

《并行计算与 GPU 编程》

课程大作业报告

Jacobi 集并行 CUDA 代码实现及思路分析



专业： 计算机科学与技术

班级： 2023211311

姓名： 金介然、丁乐航、张鑫溢

学号： 2023211199、2023211203、2023211202

目录

Jacobi 集并行 CUDA 代码实现及思路分析.....	3
1. 论文翻译（第一部分）	3
2. CUDA 并行核心代码（第二部分）	19
2.1 GPU 输入输出结构	19
2.2 GPU 端 lower-link 判定函数	19
2.3 CUDA kernel：一线程处理一条内部边	20
2.4 GPU 调用封装	21
2.5 CPU SoS 回退核心代码	22
3. 代码实现思路分析（第三部分）	22
3.1 任务背景与代码目标	22
3.2 串行老项目分析	23
3.3 论文算法理解与本项目问题映射	28
3.4 mycode 并行代码与串行代码思路的区别	30
3.5 mycode 代码详细分析	34
3.6 并行化实现的合理性与局限	48
3.7 测试与结果验证	50
实验总结.....	63

Jacobi 集并行 CUDA 代码实现及思路分析

1. 论文翻译（第一部分）

多元 Morse 函数的 Jacobi 集

(Jacobi Sets of Multiple Morse Functions)

Herbert Edelsbrunner and John Harer

摘要

定义在公共 d 维流形上的两个 Morse 函数的 Jacobi 集，是其中一个函数在另一个函数的水平集上的限制的临界点的集合。等价地，它是这两个函数的梯度平行的点的集合。对于通有的 Morse 函数对，Jacobi 集是一个光滑嵌入的 1 维流形。我们提供了一个多项式时间算法，用于计算顶点处指定了函数值的三角剖分上 Jacobi 集的分段线性对应物，并将所有结果推广到多于两个但最多 d 个 Morse 函数的情况。

关键词： 微分与计算拓扑，Morse 函数，临界点，水平集，Betti 数，算法。

1 引言 (Introduction)

本文是对多元 Morse 函数、特别是其 Jacobi 集的数学与算法研究。正如我们将看到的，该集合与多元微积分中的拉格朗日乘子法相关，我们的算法可以被视为该方法的离散对应物。

动机 (Motivation)

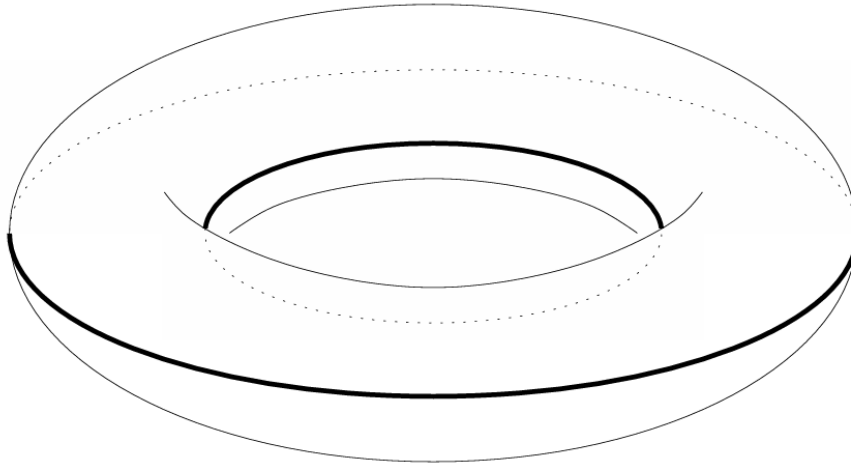
自然现象经常使用连续函数来建模，在科学领域中，在同一个区域上定义两个或多个此类函数是相当常见的场景。例如，在海洋学中，研究人员研究海水的各种属性分布，目的是阐明海洋动力学并深入了解全球气候变化 [4]。其中一个属性是温度，另一个是盐度（海水密度的重要指标）。温度分布通常在恒定盐度的层内进行研究，因为海水倾向于沿着这些层混合，而不是在层之间混合。在数学上，我们可以将温度和盐度视为公共三维空间部分上的两个连续函数。一个层由盐度函数的水平面决定，我们对限制在该表面上的温度函数感兴趣。这是二维区域上的连续函数，其临界点通常是极小值、鞍点和极大值。在本文中，我们研究当盐度值变化时这些临界点所经过的路径。事实证明，这些路径也是如果我们把盐度函数限制在温度函数的水平面上时，盐度函数的临界点所经过的路径。更一般地，我们通过分析水平集限制内的临界点，来研究定义在公共流形上的连续函数之间的关系。

有时，构建辅助函数来研究给定函数的性质是有用的。例如，考虑一个随时间变化的函数，例如由太阳、行星和太阳系中的卫星产生的引力势 [18]。在该引力势的临界点处，引力处于平衡状态。行星和卫星相对于彼此以及太阳运动，这意味着临界点会运动、出现和消失。为了研究这样一个随时间变化的函数，我们引入另一个函数，其在时空任何点的值就是时间。那么引力势临界点的路径就是定义在公共时空部分上的这两个函数的 Jacobi 集。

结果 (Results)

本文的主要研究对象是公共 d 维流形上的 $k + 1 \leq d$ 个 Morse 函数的 Jacobi 集 $J = J(f_0, f_1, \dots, f_k)$ 。根据定义，这是 f_0 限制在 f_1 到 f_k 的水平集的交集上的临界点集合。我们注意到 J 在这 $k + 1$ 个函数中是对称的，因为它是这 $k + 1$ 个梯度向量线性相关的点集。

在最简单的非平凡情况下，我们在公共二维流形上有两个 Morse 函数。在这种情况下，Jacobi 集 $J = J(f, g) = J(g, f)$ 一般来说是一组两两不相交且没有任何自交的光滑曲线。图 1 展示了二维环面上两个 Morse 函数的这一概念。通有的 $k + 1$ 个 Morse 函数集合的 Jacobi 集是一个 k 维子流形，前提是 $d > 2k - 2$ 。该不等式第一次失效是在 $d = k + 1 = 4$ 的情况下。在这种情况下，除了一个离散点集外，Jacobi 集是一个 3 维流形。



(图 1 说明：两个部分加粗和部分虚线表示的经向圆构成了 $f, g: M \rightarrow R$ 的 Jacobi 集，其中 M 是环面，并且 f 和 g 将点 $x \in M$ 映射到其在平行于经线的平面上的正交投影的笛卡尔坐标。)

我们描述了一种算法，用于计算从顶点处给定的值插值得到的 $k + 1$ 个分段线性函数的 Jacobi 集的近似。在缺乏光滑性的情况下，它模拟了通有性和可微性，并将 J 计算为

三角剖分的子复形。该算法在本质上是组合的（而非数值的），并将计算简化为测试三角剖分中 k -单纯形的临界性。一个单纯形是否被认为是临界取决于其局部拓扑。通过使用 Betti 数来表达该拓扑，我们得到了一种适用于三角剖分流形的算法，其运行时间在单纯形数量上是多项式阶的。假设链路（links）的大小由一个常数界定，运行时间与单纯形的数量成正比。

相关的先前工作 (Related prior work)

本文的工作属于光滑映射奇点的一般领域，该领域由 Hassler Whitney 在大约半个世纪前开创；参见例如 [19]。在这项工作中，从 d 维流形 M^d 到 $(k + 1)$ 维流形 N^{k+1} 的映射的褶皱（fold）是偏导数矩阵不具有满秩的点集的像。在本文中，我们考虑 Morse 函数的 $(k + 1)$ 元组，它们是特殊的光滑映射，其值域是 $(k + 1)$ 维欧几里得空间 R^{k+1} 。这种特殊映射的褶皱就是 Jacobi 集的像。我们故意将其限制在欧几里得空间中，因为这为我们的算法提供了所需的框架。

Whitney 考虑了曲面之间映射的情况，并研究了 d 相对于 k 较小的映射。微分拓扑学中的一个经典定理是 Whitney 嵌入定理，该定理指出 n 维闭合可定向流形总是可以嵌入到 R^{2n} 中。Thom 扩展了 Whitney 的工作，研究了射流（jets）空间（如果两个函数在 p 处的 k 阶或更低阶偏导数相等，则它们在 p 处具有相同的 k -射流）。Thom 横截性定理与 M 上 C^∞ 函数的 Thom-Boardman 分层一起，给出了通有函数奇点的表征 [11]。Mather 从更代数几何的角度研究了奇点，并证明了一个映射的几种稳定性定义的等价性（本工作未使用此概念）。特别地，他提供了适当的框架来从其在 Thom-Boardman 分层的层上的限制中重构映射。

在完全不同的背景下，Nicola Wolpert 使用 Jacobi 曲线开发了用于在 R^3 中求二次曲面交集的精确且高效的算法 [20]。她通过将问题简化为计算投影到 R^2 的相交曲线的排列来实现这一点。任何此类曲线都可以写成光滑函数 $R^2 \rightarrow R$ 的零集，并且任何对都定义了一条曲线，即这两个函数的 Jacobi 集。Wolpert 将它们称为 Jacobi 曲线，并利用它们来解决切向和几乎切向的交集。为了与其术语保持一致，我们决定修改该名称，并将我们更一般的概念称为 Jacobi 集。

大纲 (Outline)

第 2 节回顾了微分拓扑和组合拓扑的背景材料。第 3 节引入了两个 Morse 函数的 Jacobi 集。第 4 节描述了一种计算分段线性数据这一集合的算法。第 5 节将结果推广到三个或更多个 Morse 函数。第 6 节讨论了一小部分应用。第 7 节总结了本文。

2 背景 (Background)

本文包含针对光滑函数和分段线性函数的研究结果。为了设计一个适用于分段线性数据的算法，我们需要来自于光滑函数的 Morse 理论 [15, 16] 背景知识，以及来自于组合和代数拓扑 [1, 17] 的背景知识。

Morse 函数 (Morse functions)

设 M 为一个无边界的光滑且紧致的 d 维流形。光滑映射 $f: M \rightarrow R$ 在流形上一点 x 处的微分是一个线性映射 $df_x: TM_x \rightarrow R$ ，它将 x 处的切空间映射到 R 。如果 df_x 是零映射，则点 $x \in M$ 是临界点 (critical)；否则，它是正则点 (regular)。设 $\langle \cdot, \cdot \rangle$ 为一个黎曼度量，即切空间中的内积，它在 M 上光滑变化。由于 TM_x 中的每个向量都是通过 x 的曲线 c 在 M 中的切向量，因此梯度 ∇f 可以通过以下公式定义：对于每个 c ，有 $\langle \partial c / \partial t, \nabla f \rangle = \partial(f \circ c) / \partial t$ 。总是可以选择坐标 x_i ，使得切向量 $\frac{\partial}{\partial x_i}(x)$ 关于黎曼度量是标准正交的。对于这样的坐标，梯度由以下熟知的公式给出：

$$\nabla f(x) = \left[\frac{\partial f}{\partial x_1}(x), \frac{\partial f}{\partial x_2}(x), \dots, \frac{\partial f}{\partial x_d}(x) \right]$$

我们在局部坐标下计算 f 的 Hessian 矩阵：

$$H_f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2}(x) & \dots & \frac{\partial^2 f}{\partial x_d \partial x_1}(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_1 \partial x_d}(x) & \dots & \frac{\partial^2 f}{\partial x_d^2}(x) \end{bmatrix}$$

它是切空间 TM_x 上的一个对称双线性型。如果 Hessian 矩阵在临界点 p 处是非奇异的，则该临界点是非退化的。Morse 引理指出，在非退化临界点 p 附近，可以选择局部坐标使得函数具有以下形式：

$$f(x_1, \dots, x_d) = f(p) \pm x_1^2 \pm \dots \pm x_d^2$$

负号的个数是 p 的指标 (index)；它等于 $H_f(p)$ 负特征值的数量。这些局部坐标的存在意味着非退化临界点是孤立的。如果满足以下条件，则函数 f 被称为 Morse 函数：(i) 它的所有临界点都是非退化的，且 (ii) 对于所有临界点 $p \neq q$ ，有 $f(p) \neq f(q)$ 。

横截性与分层 (Transversality and stratification)

设 $F: X \rightarrow Y$ 是两个流形之间的光滑映射，并设 $U \subset Y$ 是一个光滑子流形。如果对于每个 $y \in U$ 且每个 $x \in F^{-1}(y)$ ，我们都有 $dF_x(TX_x) + TU_y = TY_y$ ，则称映射 F 与 U 横截 (transversal)。换句话说， X 在 x 处的切空间在导数映射下的像的基向量，与 U 在 $y = F(x)$ 处的切空间的基向量在一起，张成了 Y 在 y 处的切空间。微分拓扑学的横截性定理断言，如果 F 与 U 横截，那么 $F^{-1}(U)$ 是 X 的一个光滑子流形，且 $F^{-1}(U)$ 在 X 中的余维度与 U 在 Y 中的余维度相同 [12]。

连接两个 Morse 函数的连续族必然会经历一些过渡，在这些过渡处，函数会违反 Morse 函数的条件 (i) 和 (ii)。我们对不可避免的最少违反次数感兴趣。为此，考虑光滑函数 $M \rightarrow R$ 的无限维希尔伯特空间 $C^\infty(M)$ 。存在一个分层 $C^\infty(M) = C_0 \supset C_1 \supset C_2$ ，其中 $C_0 - C_1$ 是 Morse 函数的集合， $C_1 - C_2$ 是恰好违反条件 (i) 或 (ii) 一次的函数的集合，而 C_2 是其余函数的集合。集合 $C_0 - C_1$ 是余维度为 0 的子流形， $C_1 - C_2$ 是余维度为 1 的子流形，而 C_2 的余维度为 2。作为我们如何使用分层的说明，取两个 Morse 函数 f 和 g ，并考虑函数的 1-参数族 $h_\lambda = f + \lambda g: M \rightarrow R$ 。我们可以对其中一个函数进行微扰，使得：

- f 和 $f + g$ 都是 Morse 函数，
- 所有的 h_λ 都属于 $C_0 - C_2$ ，并且
- 该 1-参数族与 $C_1 - C_2$ 横截。

由此可知， h_λ 仅在一组离散的 λ 取值时属于 $C_1 - C_2$ 。我们稍后将使用这一点来证明某些映射的横截性，从而使我们能够应用横截性定理。

三角剖分 (Triangulations)

k -单纯形 (k -simplex) 是 $k + 1$ 个仿射独立点的凸包。给定两个单纯形 τ 和 σ ，如果 τ 是 σ 的一个面，我们记作 $\tau \leq \sigma$ 。单纯形复形 (simplicial complex) 是一个由单纯形组成的有限集合 K ，它在面关系下是封闭的，并且任意两个单纯形的交集要么为空，要么是它们共同的面。 K 的子复形是指其自身也是单纯形复形的子集。子集 $L \subseteq K$ 的闭包 (closure) 是包含 L 的最小子复形 $Cl L \subseteq K$ 。单纯形 $\tau \in K$ 的星结构 (star) 是包含 τ 的单纯形的集合，而链路 (link) 是星结构的闭包中与 τ 不相交的面组成的集合：

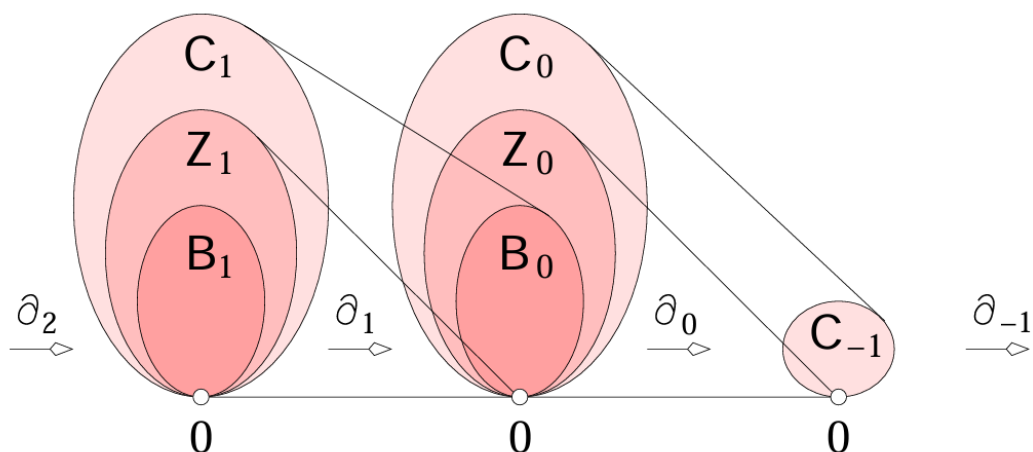
$$\begin{aligned} St \sim \tau &= \{\sigma \in K \mid \tau \leq \sigma\} \\ Lk \sim \tau &= \{v \in Cl St \sim \tau \mid v \cap \tau = \emptyset\} \end{aligned}$$

K 的顶点集记为 $Vert K$ 。底层空间 (underlying space) 是单纯形的并集： $|K| = \bigcup_{\sigma \in K} \sigma$ 。单纯形 σ 的内部是指属于 σ 但不属于 σ 的任何真面的点的集合。注意， $|K|$ 中的每个点都恰好属于 K 中某一个单纯形的内部。我们通过分段线性连续函数在顶点处的值来指定它。为了描述这种构造，设 $\varphi: Vert K \rightarrow R$ 是定义在顶点集上的函数。 $|K|$ 中的每个点 x 都有重心坐标 $\alpha_u = \alpha_u(x) \in R$ ，满足 $\sum \alpha_u u = x$ ， $\sum \alpha_u = 1$ ，并且除非 u 是其内部包含 x 的那个单纯形的顶点，否则 $\alpha_u = 0$ 。 φ 的线性延伸是定义为 $f(x) = \sum_u \alpha_u(x) \varphi(u)$ 的函数 $f: |K| \rightarrow R$ 。它是连续的，因为映射 α_u 是连续的。

如果存在同胚 $|K| \rightarrow M$ ，则单纯形复形 K 是流形 M 的一个三角剖分。在这种情况下，每个 k -单纯形的链路都对一个 $d - k - 1$ 维的球面进行三角剖分。我们将利用链路的子复形，将临界点的概念从光滑函数推广到分段线性函数。假设对于 K 中的任意两个顶点 $u \neq v$ ，有 $f(u) \neq f(v)$ ，那么 u 的下星结构 (lower star) 由星结构中以 u 为其最高顶点的所有单纯形组成，而下链路 (lower link) 是作为下星结构边界的那部分链路。注意，链路和下链路总是单纯形复形。

同调群与 Betti 数 (Homology groups and Betti numbers)

在分段线性情况下，我们使用下链路的拓扑来定义一个顶点是否被视为临界点，并且我们使用 Z_2 系数的同调群的秩来表达该拓扑。为了解释这一点，设 K 为一个单纯形复形。一个 k -链 (k -chain) 是 k -单纯形的一个子集，两个 k -链相加意味着取它们的对称差。 k -链的集合与加法运算一起构成了 k -链群，记作 C_k 。一个 k -单纯形的边界是它的 $(k-1)$ -单纯形的集合。这定义了一个边界同态 (boundary homomorphism) $\partial_k: C_k \rightarrow C_{k-1}$ ，它是通过将一个 k -链映射到其包含的 k -单纯形的边界之和而获得的。由于我们的目标是简约同调群 (reduced homology groups)，我们通过添加 $C_{-1} = Z_2$ 并将顶点的边界定义为 $\partial_0(u) = 1$ ，来扩展非平凡链群的列表。边界同态将这些链群连接起来，如图 2 所示。



(图 2 说明：由链群和连接它们的边界同态构成的链复形。)

一个 k -圈 (k -cycle) 是一个边界为零的 k -链，而一个 k -边缘 (k -boundary) 是一个 $(k+1)$ -链的边界。相应的群是 k -链群的嵌套子群： $B_k \leq Z_k \leq C_k$ 。第 k 个简约同调群是由 k -圈和 k -边缘定义的商群： $\widetilde{H}_k = Z_k/B_k$ 。第 k 个简约 Betti 数是第 k 个简约同调群的秩： $\widetilde{\beta}_k = \text{rank } \widetilde{H}_k$ 。由于我们是在模 2 下相加，所有的群都是有限的，且 $\widetilde{\beta}_k$ 是 $\widetilde{H}_k$ 的大小的二进制对数（以 2 为底的对数）。简约同调群与更常见的非简约版本的区别仅在 0 维和 -1 维上。具体来说， $\widetilde{\beta}_0$ 比连通分支的数量小 1（除非复形为空，此时 $\widetilde{\beta}_0 = 0$ ）；且除非复形为空（此时 $\widetilde{\beta}_{-1} = 1$ ），否则 $\widetilde{\beta}_{-1} = 0$ 。

倾向于使用简约同调群而非非简约同调群的主要原因，是它们与球面有着更简单的对应关系。 $(d-1)$ 维球面是 d 维欧几里得空间中与原点单位距离的点的集合： $S^{d-1} = \{x \in R^d \mid |x| = 1\}$ 。为了对 S^{d-1} 进行三角剖分，我们可以取一个 d -单纯形的真面 (proper faces) 的集合。注意， S^{-1} 是空集，它由空复形进行三角剖分。 $(d-1)$ 维球面只有一个

非零的简约 Betti 数，即 $\widetilde{\beta}_{d-1} = 1$ ，这对于所有 $d \geq 0$ 都是成立的。相比之下，一个点的简约 Betti 数全部消失（均为 0）。

3 两个函数的 Jacobi 集 (Jacobi Sets of Two Functions)

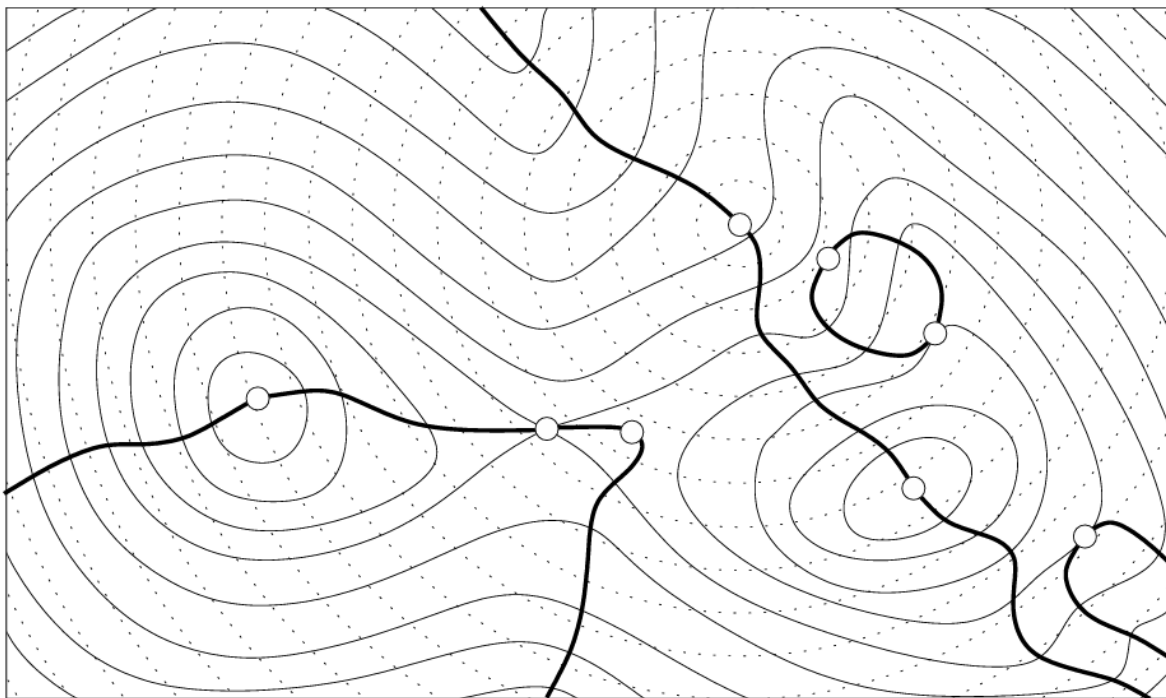
在本节中，我们考虑定义在同一流形上的一对 Morse 函数的 Jacobi 集。

Jacobi 集的定义 (Definition of Jacobi sets)

设 M 是一个光滑的 d 维流形，并选择一个黎曼度量以便我们能够定义梯度。我们假设 $d > 2$ ，并考虑两个通有的 Morse 函数 $f, g: M \rightarrow R$ 。我们对 f 限制在 g 的水平集上的性质感兴趣。对于一个正则值 $t \in R$ ，水平集 $M_t = g^{-1}(t)$ 是一个光滑的 $(d-1)$ 维流形，且 f 在该水平集上的限制是一个 Morse 函数 $f_t: M_t \rightarrow R$ 。Jacobi 集 $J = J(f, g)$ 是此类水平集限制的临界点集合的闭包：

$$J = \text{cl}\{x \in M \mid x \text{ 是 } f_t \text{ 的临界点}\}, \quad (1)$$

对于某个正则值 $t \in R$ 成立。闭包操作将 f 限制在临界值 t 处的水平集上的临界点，以及 g 的临界点（它们在这些水平集中构成了奇点）添加进了来。图 3 通过展示定义在二维平面上的两个光滑函数的 J 来说明这一定义。可以将这幅图想象成一个由 f 的（虚线）等高线（水平曲线）所表示的锥状山峰，并想象一个动画过程，在此过程中 g 的（实线）等高线滑过这座山。例如，在左侧，我们看到一个圆在一个斜坡上向外扩展，并且我们观察到一个极大值向上移动，以及一个极小值从起始点（它是 g 的一个极小值）向下移动。



(图 3 说明：函数 f 和 g 分别由它们的虚线和实线水平曲线表示。Jacobi 集用加粗实线绘制。图中标记了诞生-消亡点 (birth-death points) 以及这两个函数的临界点。)

考虑这两个函数在一点 x 处的梯度： $\nabla f(x), \nabla g(x) \in R^d$ 。设 $t = g(x)$ 。那么 f_t 在 $x \in M_t = g^{-1}(t)$ 处的梯度就是 $\nabla f(x)$ 在 M 于 x 处的切空间上的投影。由此可知， $\nabla f_t(x) = 0$ 当且仅当这两个梯度向量平行。换句话说，

$$J = \{x \in M \mid \nabla f(x) + \lambda \nabla g(x) = 0 \text{ 或 } \lambda \nabla f(x) + \nabla g(x) = 0\} \quad (2)$$

对于某个 $\lambda \in R$ 成立。式 (2) 中的第一个关系式遗漏了 $\nabla g(x) = 0$ 且 $\nabla f(x) \neq 0$ 的情况。第二个关系式存在的唯一原因就是为捕捉这些情况。式 (2) 中对 J 的描述在 f 和 g 之间是对称的： $J(f, g) = J(g, f)$ 。因此，我们将 J 的点称为 f 和 g 的同时临界点

(simultaneous critical points)。注意， $\nabla f + \lambda \nabla g$ 是函数 $f + \lambda g$ 的梯度。因此，式 (2) 暗示了 Jacobi 集的另一种表征方式：

$$J = \{x \in M \mid x \text{ 是 } f + \lambda g \text{ 或 } \lambda f + g \text{ 的临界点}\}, \quad (3)$$

对于某个 $\lambda \in R$ 成立。这一公式应当与拉格朗日乘子法进行对比，在拉格朗日乘子法中， λ 被视为一个未定元。

临界曲线 (Critical curves)

一般地， f_t 拥有一组离散的临界点，随着 t 的变化，这些点扫出 J 。由此可知， J 是一个一维集合。我们加强这一观察，并证明 Jacobi 集是 M 中一个光滑嵌入的 1 维流形。这得益于 [11] 中描述的 Jacobi 集的分层，我们将在第 5 节中对其进行简述。为了完整性，我们给出一个直接的证明，从而避免为了证明更一般的结果而需要用到的更高级概念。

光滑嵌入定理 (SMOOTH EMBEDDING THEOREM)

一般地，两个 Morse 函数 $f, g: M \rightarrow R$ 的 Jacobi 集是 M 中一个光滑嵌入的 1 维流形。

证明 (PROOF)

假设 M 是一个 d 维流形 (对于 $d \geq 2$)，并考虑函数 $F, G: M \times R \rightarrow R^d$ ，它们将点 $x \in M$ 和参数 $\lambda \in R$ 映射为 $f + \lambda g$ 和 $\lambda f + g$ 的梯度：

$$\begin{aligned} F(x, \lambda) &= \nabla f(x) + \lambda \nabla g(x), \\ G(x, \lambda) &= \lambda \nabla f(x) + \nabla g(x). \end{aligned}$$

根据式 (2)，一个点 x 属于 J 当且仅当存在一个 $\lambda \in R$ 使得 $F(x, \lambda) = 0$ 或 $G(x, \lambda) = 0$ 。令 $\Gamma_F = F^{-1}(0)$ 且 $\Gamma_G = G^{-1}(0)$ ，我们通过投影到 M 来得到 J ：

$$\begin{array}{ccc} \Gamma_F & \subseteq & M \times R \\ \downarrow & & \downarrow \\ J_F & \subseteq & M \end{array} \quad \text{and} \quad \begin{array}{ccc} \Gamma_G & \subseteq & M \times R \\ \downarrow & & \downarrow \\ J_G & \subseteq & M. \end{array}$$

我们有 $J = J_F \cup J_G$ ，其中 J_F 是 J 减去满足 $\nabla g(p) = 0$ 的离散点集，而 J_G 是 J 减去满足 $\nabla f(p) = 0$ 的点集。我们证明 F 与 0 横截，或者等价地，对于每个 $(p, \lambda) \in F^{-1}(0)$ ， F 在 (p, λ) 处的导数具有秩 d 。我们计算 $dF_{(p, \lambda)} = [H_{f+\lambda g}(p), \nabla g(p)]$ ，其中 Hessian 矩阵是一个 $d \times d$ 的矩阵。

正如第 2 节中所提到的，我们可以假设所有函数 $h_\lambda = f + \lambda g$ 都属于 $C_0 - C_1$ ，除了在一组离散的值处它们属于 $C_1 - C_2$ 。前者仅包含非退化临界点，因此 Hessian 矩阵本身就已经具有秩 d 。设 λ_0 是使得 h_{λ_0} 不是 Morse 函数的一个值。如果有两个临界点共享相同的函数值，那么 Hessian 矩阵仍然是可逆的，无须再证明其他内容。否则，存在单个诞生-消亡点 p_0 ，我们记 $c_0 = h_{\lambda_0}(p_0)$ 。存在局部坐标使得 $p_0 = (0, 0, \dots, 0)$ ， $\lambda_0 = 0$ ，并且在 (p_0, λ_0) 的邻域内有：

$$h_\lambda(p) = c_0 + x_1^3 - \lambda x_1 \pm x_2^2 \pm \dots \pm x_d^2 = \lambda(-x_1) + (c_0 + x_1^3 \pm x_2^2 \pm \dots \pm x_d^2)$$

注意，这隐含了 $g(p) = -x_1$ 。我们可以明确地写出 Hessian 矩阵和梯度，从而得到：

$$dF_{(p_0, \lambda_0)} = \begin{bmatrix} 0 & 0 & \dots & 0 & -1 \\ 0 & \pm 2 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 0 & \pm 2 & 0 \end{bmatrix}$$

该矩阵的秩为 d 。由于 $0 \in \mathbb{R}^d$ 的余维度为 d ，横截性定理现在意味着 $\Gamma_F = F^{-1}(0)$ 是 $M \times \mathbb{R}$ 中的一个光滑 1 维流形。我们仍需证明该 1 维流形的投影光滑地嵌入在 M 中。设 $\pi: \Gamma_F \rightarrow J$ 定义为 $\pi(p, \lambda) = p$ 。我们展示 π 是一对一的，且对于所有 $(p, \lambda) \in \Gamma_F$ ，有 $d\pi_{(p, \lambda)} \neq 0$ 。如果 $\pi(p, \lambda) = \pi(p', \lambda')$ ，则 $p = p'$ 且 $\nabla f(p) + \lambda \nabla g(p) = \nabla f(p) + \lambda' \nabla g(p) = 0$ 。由于在 Γ_F 上 $\nabla g(p) \neq 0$ ，因此 $\lambda = \lambda'$ ，所以该投影是单射。为了证明 π 的导数 $d\pi$ 非零，请注意 Γ_F 在 (p, λ) 处的切线是 $dF_{(p, \lambda)} = [H_{f+\lambda g}(p), \nabla g(p)]$ 的核 (kernel)。如果 $d\pi$ 在 (p, λ) 处为零，那么这条切线必须是垂直的，由向量 $v = (0, \dots, 0, 1)$ 张成。但是由于 $\nabla g(p) \neq 0$ ， v 不可能在 $dF_{(p, \lambda)}$ 的核中。

由对称性，我们为 F 证明的一切结论同样适用于 G 。这完成了我们关于“一般来说， J 是 M 中一个光滑嵌入的 1 维流形”的证明。

4 算法 (Algorithm)

在本节中，我们描述一种算法，用于从两个 Morse 函数的近似函数中计算出它们 Jacobi 集的近似。我们首先阐述我们的一般思路，随后详细描述该算法。

一般方法 (General approach)

在应用中，我们永远无法获得光滑函数，而通常只能获得近似光滑函数的非光滑函数。我们选择不将这些非光滑函数视为光滑函数的近似。恰恰相反，我们认为这些非光滑函数

是由光滑函数所近似的。这样一来，相对简单的光滑情况就成为了设计算法的指导直觉，该算法构建了人们可以称之为非光滑函数的 Jacobi 集。更具体地说，设 K 是 d 维流形 M 的一个三角剖分，并设 $\varphi, \psi: \text{Vert } K \rightarrow \mathbb{R}$ 是定义在顶点上的两个函数。我们通过对 φ 和 ψ 的分段线性延伸，获得 $f, g: |K| \rightarrow \mathbb{R}$ 。我们设想这两个分段线性函数都是一系列光滑函数序列的极限： $\lim_{n \rightarrow \infty} f_n = f$ 且 $\lim_{n \rightarrow \infty} g_n = g$ 。对于每个 n ，Jacobi 集 $J_n = J(f_n, g_n)$ 都是完全定义明确的，我们的目标是将 f 和 g 的 Jacobi 集构建为 J_n 的极限。在此过程中，我们将采取一些自由度来消除关于这究竟意味着什么的歧义。作为一项指导原则，我们消除歧义的方式要与在设想的 f 和 g 的光滑近似中出现的相应光滑概念保持一致。我们将这一原则称为可微性模拟（simulation of differentiability），并将其与简易性模拟（simulation of simplicity）[9] 相结合，使我们自己置于通有光滑函数的领域内。

在构建完 f 和 g 的 Jacobi 集之后，我们可能需要将结果分解为与其光滑对应物的结构相一致的形式。最重要的是，该算法通常会构建一个 K 的一维子复形 J ，其中的边具有正整数重数（multiplicities）。我们将证明， J 可以被展开（unfolded）为不相交闭曲线的并集，且其中所有边的重数均为 1。换句话说，我们确实可以将 $J = |J|$ 视为一系列光滑嵌入的 1 维流形的极限。

边选择（Edge selection）

我们通过追踪 1-参数函数族 $h_\lambda = f + \lambda g$ 的临界点来计算 J 。在分段线性极限下，临界点集一般来说是一组顶点的集合。随着 λ 的变化，临界点会移动到其他顶点，并且在极限情况下，这种移动是沿着 K 的边以无限大的速度发生的。我们不通过记录临界点及其移动来构建 J ，而是将 J 构建为临界点在其上移动的边的并集。换句话说，我们为每条边确定有多少个临界点从一个端点移动到另一个端点，并将 J 设为该数量为正的边的集合。然后，我们通过将边的端点添加到该集合中，使 J 成为一个子复形。注意，我们不需要对函数族 $\lambda f + g$ 重复此构造。根据定义，Jacobi 集就是该子复形的底层空间： $J = |J|$ 。

设 uv 是 K 的一条边，并设 $\lambda = \lambda_{uv}$ 是 u 和 v 处的函数值相同的时刻。在这一时刻，我们有 $h_\lambda(u) = h_\lambda(v)$ ，因此 $\lambda_{uv} = [f(v) - f(u)]/[g(u) - g(v)]$ 。为了让一个临界点沿着边从 u 移动到 v （或反之），整条边对于 $\lambda = \lambda_{uv}$ 必须是临界的。我们通过考虑 u 的链路来表达这一条件，该链路是对 $(d-1)$ 维球面的三角剖分。设 l 是 $h_{\lambda_{uv}}$ 在该链路上的限制。

临界边引理（CRITICAL EDGE LEMMA）

边 uv 属于 J 当且仅当 v 是 l 的一个临界点。此外， uv 在 J 中的重数是 v 作为 l 的临界点的重数。

我们将描述如何判定一个顶点是否为临界点，以及如果它是临界点，如何计算其重数。对于这两个操作，只需查看 u 的链路中 v 的下链路即可。注意， v 在 u 的链路中的链路与 uv 在 K 中的链路是相同的，并且 l 在该链路的所有顶点以及 v 处都是有定义的。因此，

我们可以将 uv 的下链路构建为由函数值小于 v 的函数值的顶点 w 所导出的子复形。我们将该下链路记为 $\underline{Lk} \sim uv$ ，并注意它与 v 在 u 的链路中的下链路是相同的。这一公式显而易见地表明，对边 uv 的测试关于 u 和 v 是对称的。我们按如下方式实现“临界边引理”：

```

integer ISJACOBI(Edge uv)
 $\lambda = [f(v) - f(u)]/[g(u) - g(v)];$ 
 $\underline{Lk}uv = \{\tau \in \underline{Lk}uv \mid w \leq \tau \Rightarrow h_\lambda(w) < h_\lambda(v)\};$ 
return ISCRITICAL( $\underline{Lk}uv$ ).

```

如所述，函数 ISCRITICAL 如果在下链路 $\underline{Lk} \sim uv$ 为正则点时返回 0，否则返回该临界性的重数。

临界性与重数 (Criticality and multiplicity)

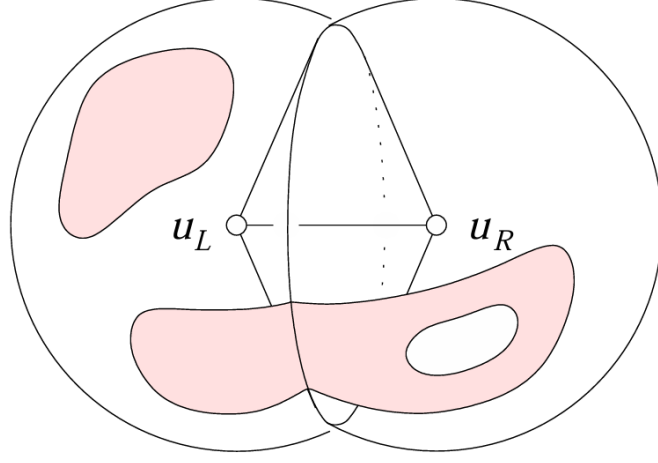
顶点链路的光滑对应物是围绕流形 M 上一点 x 绘制的足够小的球面。如果 M 是一个 d 维流形，则该球面的维度为 $d - 1$ 。下链路的光滑对应物是该球面中函数值小于或等于 x 处函数值的那部分。假设光滑性，对于所有足够小的球面，这部分的拓扑是相同的；且假设是一个 Morse 函数，它要么具有一个点的同伦型，要么具有一个球面的同伦型。在前一种情况下， x 是正则的；在后一种情况下， x 是临界的。球面的维度比 x 的指标小 1。这启发我们使用下链路的简约 Betti 数来对三角剖分中的顶点进行分类，如表 1 所示。

	$\beta \sim -1$	$\beta \sim 0$	$\beta \sim 1$	$\beta \sim 2$
正则点 (regular)	0	0	0	0
指标 0 (index 0)	1	0	0	0
指标 1 (index 1)	0	1	0	0
指标 2 (index 2)	0	0	1	0
指标 3 (index 3)	0	0	0	1

(表 1 说明：使用下链路的简约 Betti 数将顶点分类为正则点和简单临界点（通过其指标识别）。)

顶点 u 的重数是其下链路的简约 Betti 数之和： $\mu(u) = \sum_{k \geq 0} \widetilde{\beta}_{k-1}$ 。表 1 仅显示了重数为 0 的正则点，以及重数为 1 的简单临界点。所有其他点都可以被视为 $\mu \geq 2$ 个简单临界点的聚集。在低维度中，可以有效地将这样一个点展开为 μ 个简单临界点。我们以 3 维流形三角剖分中的一个顶点 u 为例来说明这一点。它的链路是一个 2 维球面。我们将下链路的连通分支称为海洋 (oceans)，将其补集的连通分支称为大陆 (continents)。因为重数至少为 2，我们有 $\mu(u) = \widetilde{\beta}_0 + \widetilde{\beta}_1 \geq 2$ 。这里有 $\widetilde{\beta}_0 + 1$ 个海洋和 $\widetilde{\beta}_1 + 1$ 个大陆。如图

4 所示，我们沿着一条穿过恰好一个海洋和一个大陆且与每个分支仅相交于单个区间的圆来切割该链路。



(图 4 说明：一个带有两个（阴影区域）海洋和两个大陆的 2 维球面被切割以形成两个 2 维球面，它们的结构在左侧对应一个简单的指标-1 临界点，在右侧对应一个简单的指标-2 临界点。)

我们将 u 替换为两个顶点 u_L 和 u_R ，并将它们连接到链路的相应两侧以及彼此连接。我们有 $\widetilde{\beta}_{0L} + \widetilde{\beta}_{0R} = \widetilde{\beta}_0$ 且 $\widetilde{\beta}_{1L} + \widetilde{\beta}_{1R} = \widetilde{\beta}_1$ 。为了确保算法能够继续推进，我们进行切割时要保证 $\widetilde{\beta}_{0L} + \widetilde{\beta}_{1L}$ 和 $\widetilde{\beta}_{0R} + \widetilde{\beta}_{1R}$ 都小于 μ 。这意味着两者都至少为 1，这等价于避免创建正则点以及极大值和极小值。我们重复该过程，直到所有顶点都成为简单临界点。这会在 $\widetilde{\beta}_0 + \widetilde{\beta}_1 - 1$ 次拆分后完成，这些拆分会产生 $\widetilde{\beta}_0$ 个指标为 1 的简单临界点和 $\widetilde{\beta}_1$ 个指标为 2 的简单临界点。总而言之，我们通过对下链路的简约 Betti 数求和来计算顶点的重数：

integer ISCRITICAL(*Complex* L)
for each $k \geq 0$ *do compute* $\widetilde{\beta}_{k-1}$ *of* L *endfor*;
return $\sum_{k \geq 0} \widetilde{\beta}_{k-1}$.

为了计算简约 Betti 数，我们可以使用 [17] 中描述的 Smith 法式算法。对于模 2 的系数，这相当于对关联矩阵进行高斯消元，这需要消耗与 L 中单纯形数量呈三次关系的时间。即使对于整数系数，最坏情况下的运行时间在单纯形数量上也是多项式阶的 [13]。对于维度 $d \leq 5$ 的流形，我们获得的下链路是维度为 $d - 2 \leq 3$ 的球面的子复形。对于这些情况，我们可以使用 [7] 中明显更快的增量算法，其运行时间仅比与链路中单纯形数量成正比的时间稍大一点。

后处理 (Post-processing)

第二步相当于将边的并集展开为一个 1 维流形。将顶点 u 的度数 (degree) 定义为 J 中共享 u 的边的数量, 其中每条边都按其重数计算。我们利用了“每个顶点的度数均为偶数”这一事实。这在我们对光滑情况下 J 的分析中得到了提示, 但需要一个直接的证明, 下面使用初等的奇偶性论证来给出该证明。对于光滑函数的良好分段线性近似, 大多数顶点的度数都将为零。如果 u 的度数为 2 或更高, 我们将成对地粘合事件的被选边。对于 $d = 2$, 这样做可以使粘合的边对在 u 处不发生交叉。

偶数度数引理 (EVEN DEGREE LEMMA) J 中每个顶点的度数都是偶数。

证明 (PROOF)

We 考虑一个顶点 u 和函数族 $h_\lambda = f + \lambda g$ 。对于 $\lambda = \pm\infty$, $\mu(u)$ 独立于 f , 且在两个极端处相同。我们将 λ 从 $-\infty$ 连续增加到 $+\infty$ 。每当我们越过一个值 $\lambda = \lambda_{uv}$ 时, 邻居 v 的状态就会在 u 的下链路内部和外部之间发生切换。这种变化对 u 状态的影响取决于 v 在 h_λ 限制于 $Lk \sim u$ 时的类型。具体来说, u 的重数要么增加、要么减少, 其改变量为 v 在 u 的下链路中的重数。例如, 如果 v 是正则的, 则该变化没有影响; 如果 v 是一个简单临界点, 则它会使 $\mu(u)$ 增加或减少 1。每次操作 (均计入 v 的重数) 的总次数等于 u 的度数。由于我们开始和结束时 u 的重数相同, 因此我们增加和减少的次数相等, 这隐含了度数是偶数。

5 $k + 1$ 个函数的 Jacobi 集 (Jacobi Sets of $k+1$ Functions)

我们将第 3 节和第 4 节的结果从定义在同一流形上的两个函数推广到三个或更多个函数。

光滑情况 (Smooth case)。设 M 是一个光滑的 d 维流形并选择一个黎曼度量。考虑 M 上的 $k + 1 \leq d$ 个 Morse 函数 $f_0, f_1, \dots, f_k$, 并记 $\Phi = (f_0, f_1, \dots, f_k): M \rightarrow R^{k+1}$ 。这些 f_i 中的 $j \leq d$ 个函数的水平集的通有交集是一个 $(d - j)$ 维的光滑流形。设 $t = (t_0, \dots, \hat{t}_l, \dots, t_k)$ 是一个由目标值组成的 k -维向量, 其中上帽标记 ($\hat{}$) 表示 t_l 不在该向量中。一般来说, 相应水平集的交集是一个光滑的 $(d - k)$ 维流形: $M_t = \bigcap_{i \neq l} f_i^{-1}(t_i)$, 并且 f_l 在 M_t 上的限制 f_{lt} 是一个 Morse 函数。我们把该限制函数的临界点称为同时临界点。Jacobi 集 $J = J(\Phi)$ 是这些同时临界点集合的闭包:

$$J = \text{cl}\{x \in M \mid x \text{ 是 } f_{lt} \text{ 的临界点}\}, \quad (4)$$

对于某个索引 l 和某个 k -维向量 t 成立。一般来说, 一点 $x \in M_t$ 处的 k 个梯度 ∇f_i 张成了在 x 处法随于 M_t 的 k 维线性子空间, 并且 x 是 f_{lt} 的临界点当且仅当 $\nabla f_l(x)$ 属于该线性子空间。这意味着 Jacobi 集的一个直接定义是:

$$J = \{x \in M \mid \text{rank } d\Phi_x \leq k\}, \quad (5)$$

其中 $d\Phi_x$ 是一个 $d \times (k+1)$ 的矩阵，其列向量为 f_0 到 f_k 在 x 处的梯度。式 (5) 在 f_i 的各个分量之间是对称的，这隐含了 J 独立于 f_i 的排列顺序。这 $k+1$ 个梯度向量的线性相关性意味着，对于每个 $x \in J$ ，存在一个梯度可以写成其他梯度的线性组合： $\nabla f_l(x) + \sum_{i \neq l} \lambda_i \nabla f_i(x) = 0$ 。由于梯度的线性组合即为相应函数组合的梯度，因此这隐含了：

$$J = \{x \in M \mid x \text{ 是 } f_l + \sum_{i \neq l} \lambda_i f_i \text{ 的临界点}\}, \quad (6)$$

对于某个索引 l 和某些参数 λ_i 成立。一般来说， J 是由一个 k -参数的离散点族所扫出的。由此可知， J 是一个维度为 k 的集合。遗憾的是， J 并不总是一个 M 的子流形。遵循 [11]，我们定义 $S_r = \{x \in M \mid \text{corank } d\Phi_x = r\}$ （注：原文为 *rank*，此处根据逻辑和上下文余秩定义对应 *corank*）。显然，对于 $r \geq 1$ ， J 是 S_r 的不相交并集。第 2 节中解释的横截性定理以及 Thom 横截性定理 [11, 54 页] 隐含了，对于每个 r ， S_r 是 M 的一个余维度为 $r(r+d-k-1)$ 的子流形，然而它不一定是闭子流形。此外，每个 S_r 的闭包是并集 $S_r \cup S_{r+1} \cup \dots$ ，但它不一定是一个流形。特别地，只要 S_2 （以及因此对于所有 $r \geq 2$ 的 S_r ）为空， $J = \text{cl } S_1$ 就是一个流形。只要对于 $r=2$ 满足 $d < r(r+d-k-1)$ ，这种情况就会发生，因为负的余维度意味着它为空。该不等式等价于 $d > 2k-2$ ，并且它第一次失效是在一个从 4 维流形到 R^4 的映射中。

算法 (Algorithm)

设 K 是 d 维流形 M 的一个三角剖分，并设 $\varphi_i: \text{Vert } K \rightarrow R$ 是定义在顶点上的函数（对于 $0 \leq i \leq k$ ）。对于每个 i ，我们获得 $f_i: |K| \rightarrow R$ 作为 φ_i 的分段线性延伸。与 $k+1=2$ 个函数的情况一样，我们将 f_i 视为光滑函数的极限，并将一个 k 维子复形 $J \subset K$ 构建为这些光滑函数的 Jacobi 集的极限。具体来说，我们计算 K 中每个 k -单纯形的重数，并将 J 定义为具有正重数的 k -单纯形的集合加上这些 k -单纯形的面的集合。根据定义，Jacobi 集就是该子复形的底层空间： $J = |J|$ 。

正如式 (6) 所提示的，我们引入一个 k -参数的函数族 $h_\lambda = f_l + \sum_{i \neq l} \lambda_i f_i$ ，其中 $\lambda = [\lambda_0, \dots, \hat{\lambda}_l, \dots, \lambda_k]$ 。接下来，我们考虑 K 中的一个具有顶点 $u_0, u_1, \dots, u_k$ 的 k -单纯形 σ ，并令 λ_σ 为满足 $h_{\lambda_\sigma}(u_0) = h_{\lambda_\sigma}(u_1) = \dots = h_{\lambda_\sigma}(u_k)$ 的 k -维向量。 σ 的链路是一个三角剖分的 $(d-k-1)$ 维球面，并且和前面一样，我们将下链路构建为由函数值小于 σ 的顶点的函数值的顶点 w 所导出的子复形。 σ 是否属于 J 取决于下链路的拓扑，更准确地说是取决于下链路的简约 Betti 数：

```

integer ISJACOBI( $k$ -Simplex  $\sigma$ )
    compute  $\lambda = \lambda_\sigma$ ;
     $\underline{Lk}\sigma = \{\tau \in \underline{Lk}\sigma \mid w \leq \tau \implies h_\lambda(w) < h_\lambda(u_0)\}$ ;
    return ISCRITICAL( $\underline{Lk}\sigma$ ).

```

函数 ISCRITICAL 与第 4 节中的相同。可能令人惊讶的是，随着函数数量的增加，临界性测试反而变得更容易了，这仅仅是因为链路的维度随着 k 的增加而减小。

6 走向应用 (6 Towards Applications)

不同的应用以不同的方式提供数据，并需要对算法进行不同的调整。为了展示本文所提出结果的广泛潜力，我们讨论了几个应用，同时强调它们引发的问题的多样性。

轮廓线 (Contours)

设 M 是 R^3 中一个光滑嵌入的 2 维流形，并设 $a \in S^2$ 为一个任意但固定的视线方向。轮廓线是指视线方向属于切平面的点 $x \in M$ 的集合： $a \in TM_x$ 。我们引入两个函数 $f, g: M \rightarrow R$ ，分别定义为 $f(x) = \langle x, b \rangle$ 和 $g(x) = \langle x, c \rangle$ ，其中 $b, c \in S^2$ 是与 a 正交的方向。假设 $c \neq \pm b$ ，则 $f(x)$ 和 $g(x)$ 是沿视线方向投影的坐标。如图 1 所示， f 和 g 的 Jacobi 集恰好是 M 的轮廓线，而轮廓线的投影就是这两个映射的褶皱 (fold)。注意， $J = J(f, g)$ 是 M 中一个光滑嵌入的 1 维流形，而褶皱则可以具有奇点，例如尖点 (cusps) 和自交点。

假设 M 是通过 R^3 中的一个近似单纯形复形 K 给出的。第 4 节中描述的算法相当于在 K 中选择满足以下条件的边：平行于 a 且穿过该边的平面将两个事件三角形都留在其同一侧。实验中已经观察到， J 在几何上近似于轮廓线，但在拓扑上则不一定。具体来说，我们可以通过细分 K 来提高近似的几何精度。在此过程中，Jacobi 集的伪圈 (spurious cycles) 会变小，但它们可能不会消失，甚至在极限情况下也不会 [5]。这种“拓扑噪声”是分段线性近似的一种产物，在双曲区域尤为常见，例如图 1 中环面的内侧经向圆周围。我们将把 [8] 中用于测量拓扑噪声的方法推广到 Jacobi 集作为一个开放性问题提出来。

蛋白质相互作用 (Protein interaction)

能量势是科学领域中常见的一种光滑函数。相应的力与势的关系，就像梯度与光滑函数的关系一样。例如，考虑在蛋白质相互作用背景下研究的力，例如静电力和范德华力。范德华势衰减迅速，可能与到源距离的六次方成反比，而静电势衰减慢得多，与距离成反比。因此，人们认为静电势在早期阶段就通过引导蛋白质相互靠近来影响相互作用，此时蛋白质还相距相对较远。一旦紧密接触，范德华力就会接管，这可能就是稳定的相互作用需要良好的局部形状互补性的原因 [10]。通过其 Jacobi 集来可视化不同能量势之间的关系，并将这一假设付诸计算测试，将是一件非常有趣的事情。也许该假设对于某些蛋白质更成立，而对于另一些蛋白质则不然。在可视化这种关系之余，量化这些势之间的符合与不符合也是非常有用的。目前尚不完全清楚如何构建这样一个测试。上述势的自然区域是 R^3 ，但使用二维区域可能会更方便，例如分子表面 [6] 和蛋白质之间的界面 [2]。

太阳系 (Solar system)

与静电势类似，天体施加的引力势衰减速率与距离成反比。在时间上的任意时刻，作用于太阳系中某一点的引力取决于其质量以及到太阳、行星和卫星的距离。设 $M = R^3 \times R$ 代表我们太阳系中的时空，并设 $g: M \rightarrow R$ 为通过将太阳、行星和卫星的贡献相加而获得的引力势。该函数除了在天体的中心处之外都是光滑的，在中心处 g 趋于无穷大。在固定时间 t 处，我们有一个从 R^3 到 R 的映射，它一般来说具有四种类型的临界点。这些点描绘出了曲线，我们将其建模为 g 与另一个函数 f 的 Jacobi 集，其中 f 将每个时空点 $(x, t) \in R^3 \times R$ 映射到其时间： $f(x, t) = t$ 。 f 的每个水平集都是在某个固定时刻的引力势，因此显而易见， $J(f, g)$ 确实是由这些临界点描绘出的 1 维流形。

NASA 在规划其航天器飞行路径时使用的拉格朗日点（Lagrange points）与这些曲线相关，但更为复杂，因为它们是根据引力与运动航天器动量之间的相互作用来定义的。看看拉格朗日点是否也可以使用 Jacobi 集的框架来进行建模，将会是一件很有意思的事情。

7 讨论 (Discussion)

本文的主要贡献是一种算法，该算法用于构建公共三角剖分流形上一组分段线性连续函数的 Jacobi 集。定义 Jacobi 集的关键概念是分段线性函数临界点的概念。我们决定使用简约 Betti 数，部分原因是为了计算上的便利和可行性。Banchoff [3] 使用的基于下链路 Euler 示性的较弱定义也是可行的，但会遗漏该集合的重要部分。而要求正则点的加厚下链路和一个球体同胚的更强定义，对于维度为 5 或更高的链路会导致一个不可判定的识别问题 [14]。

我们以本文工作引发的一系列开放性问题来结束本节。该列表不包括前面章节中已经阐述的问题。

- 第 4 节中的算法可以更一般地应用于流形之外的对象，例如同调流形（homology manifolds）。根据定义，如果我们使用同调来对链路进行分类，这些对象就无法与流形区分开来。那么，我们的算法对其有意义的单纯形复形的最广泛类别是什么？
- 初步的计算实验表明，粗糙的三角剖分会导致对 Jacobi 集的近似效果较差。这表明我们需要一种自适应网格划分方法，根据 Jacobi 集的可用信息对三角剖分进行局部加密。
- Jacobi 集的定义 (5) 可以扩展到 $k + 1 > d$ 个 Morse 函数的情况。根据 [11]， J 的余维度为 $k - d + 2$ ，因此维度为 $2d - k - 2$ 。在科学应用中， $k + 1 > d$ 个 Morse 函数的 Jacobi 集有什么意义？
- 最后，探索本文的方法是否可以扩展到更一般的向量场，特别是扩展到非梯度场的光滑向量场，将会非常有趣。

2. CUDA 并行核心代码（第二部分）

本部分只展示本次开发中与 CUDA 并行计算直接相关的核心代码，完整工程文件位于 CODE/mycode。

2.1 GPU 输入输出结构

CODE/mycode/jacobi_gpu.h（第 7–27 行）

```
struct EdgeRecord {
    int e1;
    int e2;
    int link1;
    int link2;
};

struct JacobiGpuResult {
    int fv1;
    int fv2;
    int min_f;
    int min_g;
    int pos_align;
    int is_jacobi;
    int degenerate;
};

struct GpuTiming {
    float kernel_ms;
    float total_ms;
};
```

2.2 GPU 端 lower-link 判定函数

CODE/mycode/jacobi_cuda.cu（第 43–109 行）

```
__device__ bool device_pos1(int a,
                             int b,
                             int v,
                             const double *f,
                             const double *g,
                             int *degenerate) {
    const double x =
        (f[a] * (g[v] - g[b])) +
        (f[b] * (g[a] - g[v])) +
        (f[v] * (g[b] - g[a]));

    if (!device_are_equal(x, 0.0)) {
        return x > 0.0;
    }
}
```



```

        const double *g,
        JacobiGpuResult *results) {
    const int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= edge_count) {
        return;
    }

    const EdgeRecord edge = edges[idx];
    int degenerate = 0;

    const bool f_lower_v1 = device_is_lower_link(edge.e1, edge.e2, edge.link1, f, g, g, &degenerate);
    const bool f_lower_v2 = device_is_lower_link(edge.e1, edge.e2, edge.link2, f, g, g, &degenerate);
    const bool g_lower_v1 = device_is_lower_link(edge.e1, edge.e2, edge.link1, f, g, f, &degenerate);
    const bool g_lower_v2 = device_is_lower_link(edge.e1, edge.e2, edge.link2, f, g, f, &degenerate);

    const bool f_crit = (f_lower_v1 == f_lower_v2);
    const bool g_crit = (g_lower_v1 == g_lower_v2);
    const bool is_jacobi = f_crit;

    JacobiGpuResult out;
    out.fv1 = edge.e1;
    out.fv2 = edge.e2;
    out.min_f = !f_lower_v1;
    out.min_g = g_lower_v1;
    out.pos_align = device_alignment(edge.e1, edge.e2, f, g, &degenerate);
    out.is_jacobi = is_jacobi;
    out.degenerate = degenerate || (f_crit != g_crit);
    results[idx] = out;
}

```

2.4 GPU 调用封装

CODE/mycode/jacobi_cuda.cu 中 compute_jacobi_gpu() (第 226–243 行)

```

const int block_size = 256;
const int grid_size = static_cast<int>((edges.size() + block_size - 1) / block_size);

cudaEventRecord(kernel_start);
compute_jacobi_kernel<<<grid_size, block_size>>>>(d_edges,
    static_cast<int>(edges.size()),
    d_f,
    d_g,
    d_results);
cudaEventRecord(kernel_stop);
cudaEventSynchronize(kernel_stop);
cudaEventElapsedTime(&timing->kernel_ms, kernel_start, kernel_stop);

cudaMemcpy(results->data(), d_results, edges.size() * sizeof(JacobiGpuResult), cudaMemcpyDeviceToHost);

```

2.5 CPU SoS 回退核心代码

CODE/mycode/main_cuda.cpp (第 99–138 行)

```
static size_t apply_sos_fallback(const trimesh::TriMeshJ &mesh,
                                const std::vector<double> &f,
                                const std::vector<double> &g,
                                const std::vector<EdgeRecord> &edges,
                                std::vector<JacobiGpuResult> *results) {
    JacobiSet js(&mesh, &f, &g);
    size_t fallback_count = 0;

    for (size_t i = 0; i < edges.size(); ++i) {
        if (!(*results)[i].degenerate) {
            continue;
        }

        const EdgeRecord &edge = edges[i];
        const bool f_lower_v1 = js.is_lowerLink(edge.e1, edge.e2, edge.link1, &g);
        const bool f_lower_v2 = js.is_lowerLink(edge.e1, edge.e2, edge.link2, &g);
        const bool g_lower_v1 = js.is_lowerLink(edge.e1, edge.e2, edge.link1, &f);

        const bool f_crit = (f_lower_v1 == f_lower_v2);
        JacobiGpuResult &out = (*results)[i];
        out.fv1 = edge.e1;
        out.fv2 = edge.e2;
        out.min_f = !f_lower_v1;
        out.min_g = g_lower_v1;
        out.pos_align = js.alignment(edge.e1, edge.e2);
        out.is_jacobi = f_crit;
        out.degenerate = 1;
        ++fallback_count;
    }

    return fallback_count;
}
```

3. 代码实现思路分析（第三部分）

3.1 任务背景与代码目标

本项目对应课程要求中的第 2、3 项：在理解论文 Jacobi Sets of Multiple Morse Functions 的基础上，对照串行代码 CODE/JacobiSetComputation-master，开发对应的 CUDA 并行代码，并详细分析代码实现思路。

原串行项目已经实现了二维三角网格上两个 Morse 函数的 Jacobi 集计算。具体到当前实验代码，输入是三角网格，两个函数默认取顶点坐标：

(1) $f = y$

(2) $g = x$

程序输出 Jacobi 集对应的网格边，并写成 *_jacobi.txt。本项目的并行化目标是围绕论文与串行代码中的核心计算步骤，提取适合 GPU 并行的主要计算部分：对所有内部边逐边判断其是否属于 Jacobi 集。最终设计如下：

(1) CPU 负责网格读取、拓扑预处理、边表构造和退化边 SoS 回退；

(2) GPU 负责大规模内部边的并行 Jacobi 判定；

(3) CPU 最后合并结果并写出与串行版本一致的结果文件。

该设计属于典型的 CPU-GPU 异构并行：CPU 保留网格 I/O、拓扑预处理和 SoS 回退，GPU 承担计算密集且彼此独立的核心判定循环。

代码实现体现三层对应关系。第一层是与论文思想对应：Jacobi 集由多个函数在流形上的临界关系诱导得到，不是普通几何曲线提取；第二层是与串行代码对应：并行程序保留原项目中 POS1、POS2、is_lowerLink 等核心判定逻辑；第三层是与 GPU 并行模型对应：串行算法中的逐边判定可拆分、无数据依赖，适合成批处理。本项目选择“边”为并行粒度，因为在二维三角网格、两个函数的情形下，Jacobi 集判定归约到每条内部边上的局部判断。

因此，本报告重点分析算法按边并行的依据、串行数据结构到 GPU 数组的转换方式、论文判定语义与浮点退化处理的保持方式，以及通过比对和可视化验证并行结果正确性的过程。

3.2 串行老项目分析

3.2.1 工程结构与主要模块

串行工程位于 CODE/JacobiSetComputation-master，核心文件包括：

(1) src/main.cpp

- 读取 mesh；
- 调用 need_edges() 和 need_neighbors() 准备拓扑关系；
- 构造两个函数数组 f 和 g；
- 创建 JacobiSet 对象并调用 compute()；

- 将结果写入 *_jacobi.txt。

(2) include/TriMeshJ.h 与 src/TriMeshJ.cpp

- 在 trimesh2 的 TriMesh 基础上增加边集合 edges;
- 提供 need_edges() 生成所有无向边;
- 提供 get_e_link() 查找一条边的 link 中两个对顶点。

(3) include/JacobiSet.h 与 src/JacobiSet.cpp

- 实现 Jacobi 集判定的核心逻辑;
- 包括 POS1、POS2、is_lowerLink、alignment、compute();
- 支持 USE_SOS, 在退化情况下使用 Simulation of Simplicity 进行精确符号判定。

按整体职责划分, 串行项目分为“网格拓扑层”和“Jacobi 判定层”。TriMeshJ 属于拓扑层, 不参与 Morse 函数计算和 Jacobi 集判定, 负责将三角网格整理为后续算法需要的边和邻接关系。JacobiSet 属于算法层, 在网格边、邻接和函数值已经准备好的基础上, 对每条边执行 lower-link 判定。main.cpp 属于实验驱动层, 负责串联输入文件、默认函数构造和输出文件。

上述分层为并行化提供了明确边界。拓扑层依赖 trimesh2 的复杂 C++ 数据结构, 算法层中的逐边判定只依赖少量整数索引和函数值。因此, 并行版本不直接迁移整个 TriMeshJ, 而是在 CPU 端复用拓扑层生成紧凑边表, 再将边表交给 GPU 算法层处理。

3.2.2 网格边与 link 的构造

三角网格中的每条内部边通常被两个三角形共享。若边为 (e1, e2), 它相邻的两个三角形中各有一个非边端点, 这两个点构成该边的 link, 记为 (v1, v2)。

串行工程的 TriMeshJ::need_edges() 通过遍历每个三角形的三条边, 使用 std::set<Edge> 去重后得到整张网格的边集合:

```
for(int i = 0; i < nf; i++) {
    for(int j = 0; j < 3; j++) {
        edges.insert( create_edge(faces[i][(j+1)%3], faces[i][(j+2)%3]) );
    }
}
```

TriMeshJ::get_e_link() 则通过两个端点的邻接表求公共邻居。公共邻居的前两个顶点即为该边的 link:

```
const std::vector<int> &n1 = neighbors[v1];
const std::vector<int> &n2 = neighbors[v2];

for(int i = 0; i < n1.size(); i++){
    std::vector<int>::const_iterator iter = find(n2.begin(), n2.end(), n1[i]);
    if( iter != n2.end() ){
        if( va == -1 ){
            va = n1[i];
        }
        else if( vb == -1 ){
            vb = n1[i];
            break;
        }
    }
}
```

非内部边无法形成完整的两个 link 顶点，代码返回 -1 并在后续计算中跳过该边。

link 是串行算法的核心局部拓扑信息。对二维三角网格而言，一条内部边通常被左右两个三角形共享，这两个相邻三角形中不在边上的顶点构成该边的局部邻域信息。Jacobi 判定不需要扫描整张网格，只依赖当前边的两个端点和 link 中的两个顶点，即可判断两个函数在该局部区域内是否发生临界关系。

边界边只有一个相邻三角形，link 中不能形成两个对顶点，缺少完整的局部两侧信息，不满足该判定的算法假设。因此串行代码通过 $v1 == -1 \parallel v2 == -1$ 过滤边界边，并行版本继承该设计。

3.2.3 串行 Jacobi 判定流程

串行核心入口是 JacobiSet::compute()。其逻辑如下：

1. 遍历所有边；
2. 取出边端点 e1, e2；
3. 调用 get_e_link() 得到两个 link 顶点 v1, v2；
4. 跳过边界边；
5. 分别判断 v1 和 v2 是否处在关于两个函数的 lower link 中；

6. 若满足临界条件，则将该边加入 Jacobi 集。

对应代码片段为：

```
for(std::set<trimesh::Edge>::const_iterator iter = mesh->edges.begin(); iter != mesh->edges.end(); iter++){
    int e1 = iter->first;
    int e2 = iter->second;

    std::pair<int,int> lnk = mesh->get_e_link(*iter);
    int v1 = lnk.first;
    int v2 = lnk.second;

    if(v1 == -1 || v2 == -1)
        continue;

    bool f_lower_v1 = is_lowerLink(e1, e2, v1, g);
    bool f_lower_v2 = is_lowerLink(e1, e2, v2, g);
    bool g_lower_v1 = is_lowerLink(e1, e2, v1, f);
    bool g_lower_v2 = is_lowerLink(e1, e2, v2, f);

    bool f_crit = (f_lower_v1 == f_lower_v2);
    bool g_crit = (g_lower_v1 == g_lower_v2);

    if(f_crit){
        jacobiedges.push_back(JacobiEdge(e1, e2, !f_lower_v1, g_lower_v1, alignment(e1, e2)));
    }
}
```

该循环的并行化基础是：每条边的判定只依赖该边端点、link 顶点和函数值，不依赖其他边的结果。所有边彼此独立计算，形成适合 GPU 并行的 $O(E)$ 任务，其中 E 是边数。

在算法语义上， f_lower_v1 与 f_lower_v2 分别表示 link 中两个顶点相对于边 $(e1, e2)$ 的 lower-link 分类。二者相等时，该局部结构相对于某个函数出现临界关系。代码同时计算 f_crit 和 g_crit ，并在二者不一致时输出警告，用于检查两个函数判定在离散实现中的一致性。最终 $if(f_crit)$ 将边加入 Jacobi 集，表示该边满足 Jacobi 判定条件。

串程序在找到 Jacobi 边时直接 `push_back` 到 `jacobiedges`。该写法适用于单执行流的 CPU 串程序，但不能直接迁移到 GPU，因为多个线程同时追加会产生写冲突。并行版本采用“每条边一个固定输出槽位”的方式，解决串行数据结构与并行执行模型之间的差异。

3.2.4 POS1、POS2 与 lower link 判定

串行代码中 `is_lowerLink()` 是边判定的核心。它先对三元组 (i, j, k) 排序，再结合 POS1 和 POS2 判断顶点 k 是否位于边 (i, j) 相对于某个函数的 lower link 中。

POS1 使用两个函数值构造二维平面中的方向判定：

```
double X = (f->at(a) * (g->at(v)-g->at(b))) +  
           (f->at(b) * (g->at(a)-g->at(v))) +  
           (f->at(v) * (g->at(b)-g->at(a)));
```

```
if( !are_equal(X, 0.0f) ) return (X > 0);
```

该公式在 (f, g) 函数值平面中判断三点的相对方向。当 X 非零时，符号直接决定结果；当 X 近似为 0 时，串行非 SoS 分支继续使用一系列 tie-break 条件保证结果可判定。

POS2 则用于某个单独标量函数上的大小比较：

```
return are_equal(F[a], F[b]) ? (a < b) : (F[a] < F[b]);
```

若启用 `USE_SOS`，这些比较不再使用普通浮点近似，而会调用 SoS 库中的符号函数，例如：

- (1) `sos_lambda3`
- (2) `sos_smaller`
- (3) `basic_isort3`

这保证了退化或近退化情况下结果的稳定性。

POS1 和 POS2 将连续理论中的“相对位置”和“上下关系”转化为离散可计算的布尔判定。POS1 中的 X 对应二维有向面积或行列式符号，用于判断三个顶点在 (f, g) 函数值平面中的相对方向；POS2 判断两个顶点在某一个函数上的先后关系。`is_lowerLink()` 将二者结合起来，最终判断一个 link 顶点是否属于边的 lower link。

退化问题是 Jacobi 集计算中的关键正确性问题。多个顶点函数值相等、三个点在 (f, g) 平面中共线、边两端点在某个函数上没有严格大小关系，都会导致普通浮点比较难以稳定决定符号。串行项目通过 `USE_SOS` 使用 Simulation of Simplicity，将这些退化情况转化为一致的符号扰动顺序。并行版本不在 GPU 上完整重写 SoS，而是检测这些近退化信号并交给 CPU SoS 回退，在工程复杂度和正确性之间取得平衡。

3.2.5 串行版本的瓶颈与并行化空间

串行代码的整体流程包括读取网格、建立拓扑、遍历边、写结果。并不是所有步骤都适合 GPU 并行：

- (1) 文件读取和写入是 I/O 操作，不适合放入 GPU；
- (2) trimesh2 的拓扑结构是复杂 C++ 容器，不适合直接搬到 device 端；
- (3) SoS 精确符号计算依赖第三方 CPU 库，也不适合直接移植到 CUDA。

真正适合并行化的是 `JacobiSet::compute()` 中的逐边判定部分。该部分具有以下特点：

- (1) 计算对象数量大，随边数线性增长；
- (2) 每条边计算流程相同；
- (3) 每条边之间无数据依赖；
- (4) 输出可按边下标独立写入。

因此，本项目将这部分改写为 CUDA kernel，直接并行化核心算法循环。

从复杂度上看，若网格有 E 条边，串行版本需要执行约 E 次边判定，每次判定又包含多次 `is_lowerLink()`、`POS1()` 和 `POS2()`。随着网格规模增大，这一循环成为计算主体。CPU 预处理和 I/O 同样消耗时间，但不具备同样清晰的数据并行结构。将逐边判定迁移到 GPU 后， E 次相互独立的局部判断被分散到大量 CUDA 线程中执行，从而降低核心判定部分的执行时间。

并行化依据来自算法中可并行工作量的占比，而不是代码行数占比。Jacobi 集提取的核心是判断每条内部边是否属于结果集，本项目对这一核心部分进行 CUDA 化。

3.3 论文算法理解与本项目问题映射

3.3.1 Jacobi 集的基本思想

论文 *Jacobi Sets of Multiple Morse Functions* 研究多个 Morse 函数在流形上的 Jacobi 集。Jacobi 集描述多个函数的梯度方向发生相关、退化或临界关系的位置。对于两个函数 f 和 g ，Jacobi 集对应曲面上二者等值结构发生相互临界变化的点集。

在连续情形中，Jacobi 集通常与梯度相关性有关；在离散三角网格中，算法需要用组合拓扑方式表达“临界关系”。串行代码采用的实现方式是检查每条边在其 link 上的 lower-link 关系，从而判断该边是否属于 Jacobi 集。

论文讨论更一般的多个 Morse 函数以及更高维流形情形，工程代码处理的是具有代表性的具体实例：二维三角网格上的两个标量函数。对于这一特例，Jacobi 集离散化为网格边集合。连续曲线在三角网格中不以任意曲线形式出现，而由一组满足条件的边来近似表示。

因此，本项目的输出不是普通几何绘制结果，而是离散 Jacobi 判定的结果：曲线由若干网格边组成，每条边是否被选中，都由两个函数在该边及其 link 上的组合临界关系决定。

3.3.2 从论文到二维三角网格实现

当前工程处理的情形如下：

- (1) 流形：二维三角网格；
- (2) 函数数量：两个；
- (3) 函数值：定义在顶点上；
- (4) 输出：Jacobi 集对应的边集合。

在该情形下，论文中的一般判定落到每条边的局部检查。对一条内部边 $(e1, e2)$ ，其 link 由两个顶点 $(v1, v2)$ 构成。算法检查 $v1$ 、 $v2$ 关于两个函数的 lower link 分类是否满足临界条件。

因此，串行代码中每条边只访问：

- (1) $e1, e2$
- (2) $v1, v2$
- (3) $f[e1], f[e2], f[v1], f[v2]$
- (4) $g[e1], g[e2], g[v1], g[v2]$

这组局部数据足以完成一次边判定。

该局部性是离散算法的重要优势。连续情形中的梯度关系通常依赖微分结构，而三角网格只包含顶点和边，因此算法使用函数值排序、方向符号和 lower-link 分类来表达临界性。对一条边而言， e_1, e_2, v_1, v_2 四个顶点构成最小局部环境，既包含边本身的信息，也包含边两侧三角形的信息。函数值定义在顶点上后，即可在该局部环境中完成判定。

本项目默认使用 $f=y, g=x$ ，这使 torus 示例具有较强对称性。最终 Jacobi 集表现为内外两圈闭合曲线，该形态与坐标函数选择有关。更换为其他标量场时，代码的判定流程不变，输入函数值改变，输出 Jacobi 集形状随之改变。

3.3.3 论文思想对并行化的启发

论文算法的局部性适合 CUDA 并行。每条边的 Jacobi 判定只依赖该边的局部 star/link 信息，不需要全局迭代、边之间同步或图遍历状态。因此本项目采用的 CUDA 映射为：

内部边数组 index i -> CUDA thread i

每个线程独立执行一次与串行 compute() 中相同的判定流程，从而保持论文算法本质，并利用 GPU 的大规模数据并行能力。

在 CUDA 实现中，边表中的第 i 条边对应第 i 个线程，线程读取 EdgeRecord[i] 和函数数组 f/g ，计算完成后写入 Result[i]。整个过程不需要线程间通信，也不需要 block 内共享内存同步。该并行结构简单、稳定，便于和串行结果逐项对照。

网格构建、邻接查找等步骤也存在进一步优化空间，但其并行化需要重新设计拓扑存储和不规则访问逻辑，复杂度明显高于逐边判定。按照论文算法的局部判定特征提取 edge-level parallelism，更符合本项目的核心目标。

3.4 mycode 并行代码与串行代码思路的区别

3.4.1 总体流程差异

串行版本流程：

```
读取网格
-> 建立 edges 和 neighbors
-> for 每条边:
    get_e_link
    is_lowerLink
    若满足条件则 push_back 到 jacobiedges
```

-> 写结果

并行版本流程：

读取网格

- > 建立 edges 和 neighbors
- > CPU 预处理内部边表 EdgeRecord[]
- > 将 EdgeRecord[]、f[]、g[] 拷贝到 GPU
- > CUDA kernel: 每个线程处理一条边
- > 将每条边结果拷回 CPU
- > CPU 对退化边执行 SoS 回退
- > 写结果

主要区别在于：串行版本在遍历过程中一边计算一边 push_back 结果；并行版本先为每条边准备固定下标的输入和输出数组，使每个 GPU 线程独立写入 results[i]。该方式避免了 GPU 多线程同时向同一个 vector 追加数据带来的竞争问题。

并行版本增加 CPU 预处理阶段，用于将串行项目中面向对象、容器化的数据结构转化为 GPU 友好的线性数据。串行代码在循环中临时调用 get_e_link()，CPU 适合访问 std::set、std::vector 和邻接表；GPU kernel 更适合访问连续数组。若在 kernel 中临时查找邻接关系，不仅需要把复杂拓扑结构迁移到 device，还会造成不规则访存和复杂控制流。因此，本项目把 link 计算提前放在 CPU 上完成，GPU 只接收已经整理好的边表。

该设计使并行 kernel 专注于数值判定，而不承担拓扑搜索。CPU 负责准备每个线程要处理的任务，GPU 负责批量执行这些任务。

3.4.2 数据结构差异

串行版本使用 std::set<trimesh::Edge> 存储边，遍历时动态调用 get_e_link()。该结构适用于 CPU，不适合 GPU，原因如下：

- (1) std::set 是树形结构，不适合连续内存访问；
- (2) TriMesh 和邻接容器不能直接在 device 端使用；
- (3) GPU kernel 需要简单、连续、可索引的数据。

因此并行版本新增 EdgeRecord：

```
struct EdgeRecord {  
    int e1;
```

```
int e2;  
int link1;  
int link2;  
};
```

每条内部边被展平成一个结构体，GPU 通过边下标读取 `edges[idx]`，即可得到完整局部拓扑。

并行版本还新增 `JacobiGpuResult`：

```
struct JacobiGpuResult {  
    int fv1;  
    int fv2;  
    int min_f;  
    int min_g;  
    int pos_align;  
    int is_jacobi;  
    int degenerate;  
};
```

串行版本的 `JacobiEdge` 只保存已判定为 Jacobi 集的边，GPU 版本为每条内部边保存一个结果。该设计避免并行追加问题，并便于后续统计和 SoS 回退。

该差异体现了串行程序与并行程序在输出组织上的不同模式。串行程序动态收集结果，因为结果数量事先未知；并行程序预分配与输入规模相同的结果数组，每个线程写入自己的位置。虽然该方式会多保存部分非 Jacobi 边的状态，但换来了无锁、无原子操作、易于调试的并行写入模式。

此外，全量结果数组为后续阶段提供了便利。程序统计 `degenerate_edges`、导出退化边，并在 CPU SoS 回退时按原边下标覆盖对应结果。若在 GPU 端直接压缩结果，虽然能够减少内存占用，但会引入 `prefix-sum` 或 `atomic append` 等额外并行算法，增加实现复杂度。

3.4.3 计算方式差异

串行版本：

- (1) 单线程依次遍历边；
- (2) 每遇到 Jacobi 边就 `push_back`；

(3) 若启用 SoS, 则所有相关判断都走 SoS。

并行版本:

- (1) GPU 中多个线程同时处理不同边;
- (2) 每个线程写入固定位置 results[idx];
- (3) 普通边使用 double 精度 device 判定;
- (4) 近退化边被标记后回到 CPU 端使用 SoS 精确覆盖。

该混合方案对不同任务采用合适的执行位置:

- (1) 规则、数量多、无依赖的边判定交给 GPU;
- (2) 数量少、逻辑复杂、依赖 CPU SoS 库的退化判定交给 CPU。

3.4.4 正确性策略差异

串行 SoS 版本以精确符号计算保证退化情形的稳定性。GPU 版本完整重写 SoS 会带来较大工程复杂度, 并存在引入新不一致的风险。因此本项目采用以下策略:

- 1. GPU 先执行非 SoS 分支的高性能判定;
- 2. 一旦遇到浮点近似相等或方向判定接近 0, 就标记为 degenerate;
- 3. CPU 对这些边调用串行 JacobiSet::is_lowerLink() 和 alignment() 重新计算;
- 4. 用 SoS 回退结果覆盖 GPU 结果。

因此, 最终输出与串行 SoS 版本保持一致。

该正确性策略采用“先快后准”的流程。GPU 对所有边进行快速初判, 其中绝大多数普通边的浮点符号判定稳定; 当检测到近似相等或方向接近退化时, 再转入 CPU SoS。由于退化边数量通常远小于总边数, 该策略保留了 GPU 对大规模普通边的吞吐优势, 同时避免在 CUDA 中重新实现复杂 SoS 符号系统。

该策略也是本项目区别于简单 CUDA 改写的关键: 仅迁移非 SoS 分支会在退化数据上产生与串行参考不一致的风险; 完整移植 SoS 到 GPU 又会显著增加工程复杂度。本项目采用混合回退, 使结果和串行 SoS 输出完全一致。

3.5 mycode 代码详细分析

3.5.1 目录结构

并行代码位于 CODE/mycode，主要文件如下：

(1) main_cuda.cpp

- CPU 端主程序；
- 网格读取与边表预处理；
- GPU 调用；
- CPU SoS 回退；
- 结果输出。

(2) jacobi_cuda.cu

- CUDA device 函数；
- CUDA kernel；
- host 端 CUDA 内存管理与 kernel launch。

(3) jacobi_gpu.h

- CPU/GPU 共享的数据结构；
- compute_jacobi_gpu() 接口声明。

(4) CMakeLists.txt

- 构建 CUDA 程序；
- 链接 trimesh2 和 SoS。

(5) build_cuda_wsl.sh

- WSL 下的一键构建脚本。

(6) compare_jacobi.py

- 正确性比对脚本。

(7) benchmark_jacobi.py

- 性能测试脚本。

(8) test_*.sh

- 各里程碑自动测试脚本。

(9) DEVELOPMENT_LOG.md

- 开发过程记录。

3.5.2 jacobi_gpu.h: 共享数据结构与 GPU 接口

`jacobi_gpu.h` 定义了 GPU 计算所需的核心数据结构。

`EdgeRecord` 保存一条内部边的完整局部拓扑：

```
struct EdgeRecord {  
    int e1;  
    int e2;  
    int link1;  
    int link2;  
};
```

其中：

(1) `e1`, `e2` 是边的两个端点；

(2) `link1`, `link2` 是这条边 `link` 中的两个对顶点。

这四个整数就是一条边判定所需的全部拓扑输入。相比直接把 `TriMesh` 传入 GPU，这种结构更简单、更连续、更适合 CUDA 访问。

`JacobiGpuResult` 保存每条边的输出：

```
struct JacobiGpuResult {  
    int fv1;  
    int fv2;  
    int min_f;  
    int min_g;  
    int pos_align;  
    int is_jacobi;  
    int degenerate;  
};
```

其中：

(1) `fv1`, `fv2`：结果边的两个端点；

(2) `min_f`, `min_g`, `pos_align`：与串行 `JacobiEdge` 对应的属性；

(3) `is_jacobi`：该边是否属于 Jacobi 集；

(4) degenerate: 该边是否触发近退化判断, 需要 CPU SoS 回退。

GpuTiming 保存 GPU 时间:

```
struct GpuTiming {  
    float kernel_ms;  
    float total_ms;  
};
```

compute_jacobi_gpu() 是 CPU 端调用 GPU 的统一接口:

```
bool compute_jacobi_gpu(const std::vector<EdgeRecord> &edges,  
    const std::vector<double> &f,  
    const std::vector<double> &g,  
    std::vector<JacobiGpuResult> *results,  
    GpuTiming *timing,  
    std::string *error_message);
```

该接口将 CUDA 细节封装在 .cu 文件中, 主程序只负责准备边表和函数数组。

从软件工程角度, jacobi_gpu.h 同时承担模块边界职责。CPU 侧不需要了解 kernel 的 grid/block 设置, 也不需要直接管理 device 指针; CUDA 侧不依赖 TriMeshJ 这类复杂类。二者之间只通过 EdgeRecord、函数值数组和 JacobiGpuResult 交换数据, 降低了耦合度。

该接口设计有利于后续扩展。替换输入函数来源时, 只需改变 main_cuda.cpp 中构造 f/g 的部分; 优化 kernel 或引入 device 端结果压缩时, 主要修改 jacobi_cuda.cu, 不影响网格读取逻辑。

3.5.3 main_cuda.cpp: CPU 驱动与整体流程

main_cuda.cpp 是并行程序入口。它同时支持两种模式:

```
./jacobi_cuda --preprocess-only <mesh.obj> [--dump-edges <edges.txt>]  
./jacobi_cuda <mesh.obj> [--output <jacobi.txt>] [--dump-degenerate <edges.txt>]
```

3.5.3.1 预处理模式

--preprocess-only 模式用于验证 CPU 端拓扑准备是否正确。代码会:

1. 读取 mesh;
2. 调用 need_edges();

3. 调用 `need_neighbors()`;
4. 遍历 `mesh.edges`;
5. 调用 `get_e_link()`;
6. 过滤边界边;
7. 生成 `interior_edges`。

核心代码如下：

```
trimesh::TriMeshJ mesh(mesh_file);
mesh.need_edges();
mesh.need_neighbors();

std::vector<EdgeRecord> interior_edges;
interior_edges.reserve(mesh.edges.size());

for (std::set<trimesh::Edge>::const_iterator it = mesh.edges.begin(); it != mesh.edges.end(); ++it) {
    std::pair<int, int> link = mesh.get_e_link(*it);
    if (link.first == -1 || link.second == -1) {
        continue;
    }

    EdgeRecord record;
    record.e1 = it->first;
    record.e2 = it->second;
    record.link1 = link.first;
    record.link2 = link.second;
    interior_edges.push_back(record);
}
```

该步骤是串行工程与 CUDA 工程之间的数据桥梁，将复杂的 TriMeshJ 拓扑转换为 GPU 能直接处理的扁平数组。

预处理阶段还保证 GPU 输入已经排除了边界边和非法 link。kernel 内部不再处理 -1 link 顶点，也不访问邻接表，只需假设每个 EdgeRecord 都是一条有效内部边。这减少了 device 端分支判断，降低了出错概率。

此外，预处理模式本身也是调试工具。通过 `--preprocess-only` 和 `--dump-edges`，程序在不运行 CUDA kernel 的情况下检查边表正确性。若 GPU 输出异常，可先确认输入边表是否与串行拓扑一致，从而缩小问题范围。

3.5.3.2 函数值构造

当前实验保持与串行 main.cpp 一致，使用顶点坐标构造两个函数：

```
std::vector<double> f(mesh.vertices.size());
std::vector<double> g(mesh.vertices.size());
for (size_t i = 0; i < mesh.vertices.size(); ++i) {
    f[i] = mesh.vertices[i][1];
    g[i] = mesh.vertices[i][0];
}
```

即：

$$(1) f = y$$

$$(2) g = x$$

该构造保证串行与并行版本输入完全一致。

3.5.3.3 调用 GPU

主程序通过 compute_jacobi_gpu() 调用 CUDA 代码：

```
std::vector<JacobiGpuResult> results;
GpuTiming timing;
std::string error;
if (!compute_jacobi_gpu(interior_edges, f, g, &results, &timing, &error)) {
    std::cerr << "GPU computation failed: " << error << "\n";
    return EXIT_FAILURE;
}
```

该接口返回每条内部边的判定结果，以及 GPU kernel 时间和 GPU 总时间。

GPU 调用被封装为独立函数，使主程序流程保持为“准备输入、调用计算、处理结果”。
error 字符串用于返回 CUDA API 失败原因，例如内存分配失败、拷贝失败或 kernel launch 失败，便于实验环境下定位问题。

3.5.3.4 SoS 回退

GPU 计算后，主程序会调用：

```
const size_t sos_fallback_count = apply_sos_fallback(mesh, f, g, interior_edges, &results);
    apply_sos_fallback() 会创建串行 JacobiSet 对象：
JacobiSet js(&mesh, &f, &g);
```

由于 CMake 中定义了 USE_SOS，这个对象内部会初始化 SoS 矩阵。随后程序只遍历 degenerate == 1 的边，重新执行：

```
const bool f_lower_v1 = js.is_lowerLink(edge.e1, edge.e2, edge.link1, &g);
const bool f_lower_v2 = js.is_lowerLink(edge.e1, edge.e2, edge.link2, &g);
const bool g_lower_v1 = js.is_lowerLink(edge.e1, edge.e2, edge.link1, &f);
const bool g_lower_v2 = js.is_lowerLink(edge.e1, edge.e2, edge.link2, &f);
```

然后覆盖 GPU 结果：

```
out.min_f = !f_lower_v1;
out.min_g = g_lower_v1;
out.pos_align = js.alignment(edge.e1, edge.e2);
out.is_jacobi = f_crit;
out.degenerate = 1;
```

因此，普通边使用 GPU 结果，退化边使用串行 SoS 结果。

SoS 回退不重新计算所有边，只处理 GPU 标记出的退化边。若所有边都回到 CPU SoS 计算，GPU 并行化收益会大幅降低；若完全不回退，退化情况下会损失与串行参考的一致性。当前实现将边分成两类：普通边走 GPU 快路径，退化边走 CPU 精确路径。

在 torus 测试中，退化边数量远小于总内部边数，验证了混合策略的合理性。即使存在 SoS 初始化开销，核心判定仍由 GPU 完成大部分工作。

3.5.3.5 结果输出

最终结果由 write_jacobi_edges() 写出，格式与串行版本一致：

```
JacobiSet
<edge_count>
v1 x1 y1 z1 v2 x2 y2 z2
...
```

该格式兼容已有可视化转换脚本，也便于与串行输出做文本或边集对比。

指定 --dump-degenerate 时，程序还会输出退化边列表：

```
# edge_id e1 e2 link1 link2 is_jacobi
...
```

该文件主要用于调试和说明 SoS 回退的触发情况。

3.5.4 jacobi_cuda.cu: CUDA 核心实现

jacobi_cuda.cu 是并行化的核心文件。它包括两部分：

1. device 端判定函数；

2. host 端 CUDA 调用封装。

3.5.4.1 device 版 are_equal

GPU 端使用固定阈值 $1e-12$ 判断近似相等：

```
constexpr double kEps = 1e-12;

__device__ bool device_are_equal(double a, double b) {
    double diff = a - b;
    if (diff < 0.0) {
        diff = -diff;
    }
    return diff < kEps;
}
```

与串行 `are_equal()` 类似，它用于识别浮点近似相等情况。一旦近似相等出现在关键判定中，GPU 会把该边标记为退化，交给 CPU SoS 重新计算。

3.5.4.2 device 版 POS1

`device_pos1()` 复刻串行 `POS1()` 的非 SoS 逻辑：

```
const double x =
    (f[a] * (g[v] - g[b])) +
    (f[b] * (g[a] - g[v])) +
    (f[v] * (g[b] - g[a]));

if (!device_are_equal(x, 0.0)) {
    return x > 0.0;
}
```

当 x 接近 0 时，三个点在 (f,g) 函数值平面中接近共线或存在退化情况。GPU 此时不直接采信普通浮点判定，而是：

```
*degenerate = 1;
```

并继续使用与串行非 SoS 分支一致的 tie-break 逻辑给出临时结果。该临时结果保证 kernel 继续执行，最终在 CPU SoS 回退中被覆盖。

3.5.4.3 device 版 POS2

`device_pos2()` 对单个函数值做大小比较：

```
if (device_are_equal(field[a], field[b])) {
    *degenerate = 1;
}
```

```

    return a < b;
}
return field[a] < field[b];

```

两个函数值近似相等时，同样标记为退化。

3.5.4.4 device 版 is_lower_link

device_is_lower_link() 对应串行 JacobiSet::is_lowerLink():

```

int a = i;
int b = j;
int v = k;
device_sort3(&a, &b, &v);

const int index_i = (i == a) ? 0 : ((i == b) ? 1 : 2);
const int index_j = (j == a) ? 0 : ((j == b) ? 1 : 2);

const bool is_pos1 = device_pos1(a, b, v, f, g, degenerate);
bool is_pos2 = false;
if (index_j == (index_i + 1) % 3) {
    is_pos2 = device_pos2(i, j, field, degenerate);
} else {
    is_pos2 = device_pos2(j, i, field, degenerate);
}

return is_pos1 == is_pos2;

```

这段逻辑是论文 lower-link 判定在 GPU 上的直接实现，也是并行版本保持算法一致性的关键。

在该函数中，field 参数用于区分当前是在相对于 f 还是 g 做 lower-link 判断。虽然 POS1 始终使用二维函数值 (f,g) 判断方向，但 POS2 需要根据当前限制函数选择不同的标量数组。这对应串行代码中 is_lowerLink(e1, e2, v, g) 和 is_lowerLink(e1, e2, v, f) 的交替调用。

因此，GPU 版本不是简单比较坐标大小，而是完整复刻串行 lower-link 组合判定流程，保证并行化改变的是执行方式，而不是数学判定规则。

3.5.4.5 CUDA kernel：一线程一边

并行化核心位于 compute_jacobi_kernel():

```

const int idx = blockIdx.x * blockDim.x + threadIdx.x;
if (idx >= edge_count) {
    return;
}

```

```
}  
  
const EdgeRecord edge = edges[idx];
```

每个线程取一个 EdgeRecord，然后执行与串行 compute() 对应的四次 lower-link 判定：

```
const bool f_lower_v1 = device_is_lower_link(edge.e1, edge.e2, edge.link1, f, g, g, &degenerate);  
const bool f_lower_v2 = device_is_lower_link(edge.e1, edge.e2, edge.link2, f, g, g, &degenerate);  
const bool g_lower_v1 = device_is_lower_link(edge.e1, edge.e2, edge.link1, f, g, f, &degenerate);  
const bool g_lower_v2 = device_is_lower_link(edge.e1, edge.e2, edge.link2, f, g, f, &degenerate);
```

随后计算临界条件：

```
const bool f_crit = (f_lower_v1 == f_lower_v2);  
const bool g_crit = (g_lower_v1 == g_lower_v2);  
const bool is_jacobi = f_crit;
```

最后把结果写入固定下标：

```
results[idx] = out;
```

该写法不存在多个线程同时写一个位置的问题，因此不需要原子操作。每个线程只写自己的 results[idx]，并行结构简单且稳定。

block_size 设为 256，这是常见的 CUDA 线程块配置，能够在多数 NVIDIA GPU 上取得较好的 occupancy，同时保持实现简单。grid_size 根据边数自动向上取整，保证 kernel 覆盖全部内部边。越界线程通过 if (idx >= edge_count) return; 直接退出。

该 kernel 的控制流包含若干 if 分支，但每条边执行的整体流程基本一致，适合 SIMT 执行模型。每个线程只读取少量 int 和 double，并写回一个结果结构体，内存访问模式相对清晰。当前实现使用 AoS 形式的 EdgeRecord，主要考虑代码可读性和与 CPU 预处理结果对应；进一步优化性能时，可改为 SoA 形式分别存储 e1/e2/link1/link2。

3.5.4.6 host 端 CUDA 封装

compute_jacobi_gpu() 负责 CUDA 内存管理和 kernel 调用。主要步骤包括：

1. 分配 device 内存：

```
cudaMalloc(&d_edges, edges.size() * sizeof(EdgeRecord));
cudaMalloc(&d_f, f.size() * sizeof(double));
cudaMalloc(&d_g, g.size() * sizeof(double));
cudaMalloc(&d_results, edges.size() * sizeof(JacobiGpuResult));
```

2. 将输入拷贝到 GPU:

```
cudaMemcpy(d_edges, edges.data(), ..., cudaMemcpyHostToDevice);
cudaMemcpy(d_f, f.data(), ..., cudaMemcpyHostToDevice);
cudaMemcpy(d_g, g.data(), ..., cudaMemcpyHostToDevice);
```

3. 设置线程块大小:

```
const int block_size = 256;
const int grid_size = static_cast<int>((edges.size() + block_size - 1) / block_size);
```

4. 启动 kernel:

```
compute_jacobi_kernel<<<grid_size, block_size>>>>(...);
```

5. 拷回结果:

```
cudaMemcpy(results->data(), d_results, ..., cudaMemcpyDeviceToHost);
```

6. 释放资源。

函数还使用 cudaEvent 记录 kernel_ms 和 total_ms，便于性能分析。

kernel_ms 和 total_ms 分别反映不同层面的耗时。kernel_ms 只表示 CUDA kernel 在 GPU 上执行核心逐边判定的时间；total_ms 包含 device 内存分配、host-to-device 拷贝、kernel 执行、device-to-host 拷贝等 GPU 调用相关开销。实验结果显示，kernel 时间通常远小于端到端时间，说明核心判定已经由 GPU 快速执行，整体耗时则仍受 CPU 预处理、I/O 和 SoS 回退影响。

compute_jacobi_gpu() 对 CUDA API 调用逐步检查错误。若 cudaMalloc、cudaMemcpy 或 kernel launch 失败，函数返回 false 并写入错误信息。该错误处理比忽略返回值更可靠，也便于在实验环境配置不正确时快速定位问题。

3.5.5 CMakeLists.txt 与构建脚本

CODE/mycode/CMakeLists.txt 启用了 C++ 和 CUDA:

```
project(JacobiCuda LANGUAGES CXX CUDA)
```

并加入以下源文件:

- (1) main_cuda.cpp
- (2) jacobi_cuda.cu
- (3) 串行工程的 TriMeshJ.cpp
- (4) 串行工程的 JacobiSet.cpp

同时链接:

- (1) trimesh2/lib.Linux64/libtrimesh.a
- (2) Detri_2.6.a/build/lib/libSoS.a
- (3) OpenMP::OpenMP_CXX

并定义:

```
target_compile_definitions(jacobi_cuda PRIVATE USE_SOS)
```

这保证 CPU 回退使用 SoS 分支。

build_cuda_wsl.sh 封装了构建命令:

```
cmake -S "${SCRIPT_DIR}" -B "${BUILD_DIR}" -DCMAKE_BUILD_TYPE=Release  
cmake --build "${BUILD_DIR}" -j"${nproc}"
```

执行该脚本即可在 WSL 中构建 CUDA 程序。

构建文件中同时包含串行工程的 TriMeshJ.cpp 和 JacobiSet.cpp, 因为并行版本复用原项目的拓扑和 SoS 逻辑。TriMeshJ.cpp 提供边和 link 的生成能力, JacobiSet.cpp 提供 CPU SoS 回退能力。链接 libtrimesh.a 和 libSoS.a 保证这些功能在新的 CUDA 可执行文件中可用。

CMAKE_CUDA_ARCHITECTURES 设置为 75 86，用于兼容当前 CUDA Toolkit 与 RTX 4060 环境。RTX 4060 属于更新架构，CUDA 12.0 对部分最新架构标识支持有限，因此使用兼容架构设置能够避免构建失败，并在目标 GPU 上正常运行。

3.5.6 compare_jacobi.py: 正确性比对

compare_jacobi.py 用于比较串行输出和 GPU 输出。它不直接比较文本顺序，而是读取每条 Jacobi 边的两个顶点编号，并规范化为 (min,max)：

```
a = int(parts[0])
b = int(parts[4])
if a > b:
    a, b = b, a
edges.add((a, b))
```

该方法不依赖输出顺序，只要边集合一致即可判定为正确。

脚本输出：

```
reference_edges: ...
candidate_edges: ...
match: yes
```

存在差异时，脚本输出 missing 和 extra 边，便于定位错误。

该脚本用于自动化测试，并避免并行程序验证过程过度依赖输出顺序。串行程序按 std::set 顺序遍历边，因此输出顺序稳定；GPU 程序当前也保留边下标顺序，但采用并行压缩或异步写出后，顺序会发生变化。只要 Jacobi 边集合相同，算法结果即为一致。因此，集合比较比直接文本比较更具有鲁棒性。

3.5.7 benchmark_jacobi.py: 性能测试

benchmark_jacobi.py 自动完成性能测试流程：

1. 调用 mesh_make 生成不同规模 torus；
2. 调用串行 JacobiSetComputation；
3. 调用 CUDA hybrid 版本；
4. 使用 compare_jacobi.py 的读取逻辑比对结果；

5. 写出 CSV。

核心流程为：

```
run_command([str(MESH_MAKE), "torus", str(u), str(v), str(mesh)], cwd=JS_DIR)
cpu_output, cpu_wall_ms = run_command([str(CPU_EXE), str(mesh)], cwd=JS_DIR / "build")
gpu_output, gpu_wall_ms = run_command([str(GPU_EXE), str(mesh), "--output", str(gpu_out)], cwd=SCRIPT_DIR)
```

CSV 字段包括：

- (1) vertices
- (2) edges
- (3) jacobi_edges
- (4) degenerate_edges
- (5) sos_fallback_edges
- (6) cpu_wall_ms
- (7) gpu_wall_ms
- (8) gpu_kernel_ms
- (9) gpu_total_ms
- (10) speedup_wall
- (11) match

该脚本为后续报告中的性能分析提供数据来源。

benchmark_jacobi.py 同时承担正确性和性能两类任务。每个规模不仅记录时间，还调用边集比较逻辑确认 match=yes。该设计避免只展示加速比而忽略正确性。对于并行算法，速度提升必须建立在结果正确的前提下。

脚本输出的 cpu_wall_ms 和 gpu_wall_ms 是从进程调用层面测得的端到端时间，包含文件读取、预处理和写出等开销；gpu_kernel_ms 和 gpu_total_ms 来自程序内部 CUDA event 统计。报告同时展示这两类时间，用于区分小规模端到端不加速与 kernel 本身执行较快这两类现象。

3.5.8 测试脚本设计

CODE/mycode 中还包含多个阶段性测试脚本：

(1) test_preprocess.sh

- 测试 CPU 边表预处理；
- 检查 torus 的顶点、边、面、内部边数量。

(2) test_gpu_basic.sh

- 测试 GPU 基础判定；
- 检查 Jacobi 边数是否为 64。

(3) test_degenerate_dump.sh

- 测试退化边导出；
- 检查退化边数量是否为 16。

(4) test_sos_fallback.sh

- 测试 SoS 回退；
- 检查 sos_fallback_edges。

(5) test_compare_jacobi.sh

- 测试串行与 GPU 输出比对。

(6) test_benchmark_smoke.sh

- 测试性能脚本能否生成 CSV，并保证 match=yes。

这些脚本体现分阶段验证流程：先验证输入预处理，再验证 GPU kernel，再验证退化处理，最后验证整体正确性和性能。

该分阶段测试方式降低了调试难度。若最终输出错误，单独的测试脚本可用于判断问题阶段：test_preprocess.sh 失败表示边表输入存在问题；test_gpu_basic.sh 失败表示 kernel 判定或 CUDA 调用存在问题；test_sos_fallback.sh 失败表示退化回退合并存在问题；test_compare_jacobi.sh 失败表示整体结果与串行参考不一致。相比只运行一个总测试，该方式更适合复杂算法的迭代开发。

3.6 并行化实现的合理性与局限

3.6.1 全流程 GPU 化取舍

本项目未将网格读取、邻接构建、SoS 库整体迁移到 GPU，原因如下：

- (1) 网格读取和结果输出属于 I/O；
- (2) trimesh2 内部使用复杂 C++ 容器，不适合直接传入 device；
- (3) SoS 是 CPU 符号计算库，完整移植成本高且容易引入不一致。

因此，项目采用 CPU-GPU 混合方案：

```
CPU: mesh 读取 + 拓扑预处理 + SoS 回退  
GPU: 内部边并行 Jacobi 判定
```

该方案没有削弱并行化重点，因为 Jacobi 集计算的核心循环正是逐边判定。对边数较大的网格，这一部分具有明显的数据并行特征。

在 GPU 编程中，合理的异构划分比“全部放到 GPU”更关键。GPU 擅长处理大量结构相同、控制流相对一致的数值任务，CPU 适合处理复杂分支、文件系统、动态容器和第三方库调用。本项目将拓扑整理和 SoS 留在 CPU，把重复的边判定放在 GPU，符合两类处理器的特点。

将 trimesh2 的邻接结构迁移到 GPU 需要重新设计图结构存储、邻接压缩和边界处理；将 SoS 库迁移到 GPU 需要重写大量符号计算逻辑。这些工作超出课程项目范围，并会增加与原串行代码结果不一致的风险。

3.6.2 并行化占比说明

从代码量层面，CPU 端仍然承担网格读取、预处理和回退工作；从算法结构层面，最核心的 $O(E)$ 判定循环已经被放入 CUDA kernel。

串行代码中最重要的循环是：

```
for(edge in mesh->edges) {  
    is_lowerLink(...);  
}
```

并行版本中对应为：

```
idx = blockIdx.x * blockDim.x + threadIdx.x;
edge = edges[idx];
device_is_lower_link(...);
```

因此并行化发生在算法核心，而不是仅仅并行文件处理或辅助步骤。

实验结果显示，GPU kernel 时间随着规模增大仍保持较低水平，说明逐边判定部分在 GPU 上执行效率较高。端到端加速比受 CPU 预处理、文件 I/O、数据传输和 SoS 初始化影响，没有达到很高水平。这表明当前程序属于保守可靠的 hybrid 实现；进一步优化 CPU 预处理或减少数据传输后，整体加速仍有提升空间。

3.6.3 当前实现局限

当前实现存在以下局限：

1. CPU 预处理仍是串行或依赖 trimesh2 内部实现；
2. GPU 输出结果采用全量数组，未做 device 端 compaction；
3. SoS 回退在 CPU 上执行，退化边较多时会影响端到端性能；
4. benchmark 中 gpu_wall_ms 包含 CPU 预处理、文件 I/O、SoS 初始化、数据传输等，不等同于纯 kernel 时间；
5. 当前函数默认仍是 $f=y, g=x$ ，若要支持任意外部标量场，还需扩展输入读取接口。

这些局限不影响课程要求中的“对应并行 CUDA 代码”目标，可作为进一步工作方向。

进一步优化方向包括：第一，将边表构造后的数据长期保留，避免多次 benchmark 中反复读取和预处理网格；第二，将 EdgeRecord 改为 SoA 布局，提高大规模连续访存效率；第三，在 GPU 端实现并行 compact，只输出 Jacobi 边，减少 CPU 回收结果的体积；第四，对退化边建立更精细的分类，只在真正影响最终结果时执行 SoS 回退；第五，扩展输入接口，支持从外部文件读取任意标量场，而不是固定使用 $f=y, g=x$ 。

当前实现已经形成清晰的算法框架：CPU 预处理、GPU 判定、CPU 精确回退、结果验证。后续优化主要在该框架内提升性能和通用性。

3.7 测试与结果验证

3.7.1 环境验证

环境验证的目标是确认 CUDA 编译器、NVIDIA GPU 驱动以及本项目的 CUDA 程序构建流程均可正常工作。测试在 WSL Ubuntu 环境中进行，首先进入并行代码目录：

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"
```

3.7.1.1 CUDA 编译器验证

执行：

```
nvcc --version
```

输出如下：

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2023 NVIDIA Corporation
Built on Fri_Jan__6_16:45:21_PST_2023
Cuda compilation tools, release 12.0, V12.0.140
Build cuda_12.0.r12.0/compiler.32267302_0
```

说明 WSL 中已安装 CUDA Toolkit，且 nvcc 编译器可用。本项目后续 CUDA 代码均使用该编译器构建。

3.7.1.2 GPU 运行环境验证

执行：

```
nvidia-smi
```

输出显示：

```
NVIDIA-SMI 580.105.07
Driver Version: 581.80
CUDA Version: 13.0
GPU Name: NVIDIA GeForce RTX 4060
```

WSL 能够正确识别 NVIDIA GeForce RTX 4060，说明 GPU 已经透传到 WSL 环境中。nvidia-smi 中显示的 CUDA Version 为驱动支持的最高 CUDA 运行时版本，而 nvcc --

version 显示的是实际安装的 CUDA Toolkit 版本。驱动版本高于 Toolkit 版本属于正常情况，能够向下兼容 CUDA 12.0 编译生成的程序。

进入 WSL 时出现提示：

```
wsl: 检测到 localhost 代理配置，但未镜像到 WSL。NAT 模式下的 WSL 不支持 localhost 代理。
```

该提示与 CUDA 编译运行无关，只表示 Windows 侧 localhost 代理没有自动映射到 WSL。本次测试中 nvcc、nvidia-smi 和构建过程均正常，因此该提示不影响本项目运行。

3.7.1.3 项目构建验证

执行：

```
bash build_cuda_wsl.sh
```

输出如下：

```
-- Configuring done (0.1s)
-- Generating done (0.2s)
-- Build files have been written to: /mnt/d/金介然/大三下/gpu/大作业/CODE/mycode/build
[100%] Built target jacobi_cuda
Built: /mnt/d/金介然/大三下/gpu/大作业/CODE/mycode/build/jacobi_cuda
```

构建过程成功生成 build/jacobi_cuda 可执行文件，说明 CMake、C++ 编译器、CUDA 编译器、trimesh2 静态库以及 SoS 静态库均能正确协同工作。

综上，CUDA 并行政务的开发与运行环境验证通过。

3.7.2 功能正确性验证

功能正确性验证按模块逐步进行，目标是分别确认 CPU 预处理、GPU 基础计算、退化边导出和 SoS 回退均能正常工作。测试均在 CODE/mycode 目录下执行：

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"
```

3.7.2.1 CPU 预处理验证

执行：

```
bash test_preprocess.sh
```

输出如下：

```
Reading /mnt/d/金介然/大三下/gpu/大作业/CODE/JacobiSetComputation-master/test_torus.obj... Done.
Computing edges... Done. in 0.169000 msec.
Finding vertex neighbors... Done.
mesh: /mnt/d/金介然/大三下/gpu/大作业/CODE/JacobiSetComputation-master/test_torus.obj
vertices: 512
edges: 1536
faces: 1024
interior_edges: 1536
```

该结果说明程序能够正确读取 test_torus.obj，并得到：

- (1) 顶点数：512；
- (2) 边数：1536；
- (3) 面数：1024；
- (4) 内部边数：1536。

对于该 torus 网格，所有边都是内部边，因此 interior_edges 与 edges 数量一致。该测试验证了 CPU 端边表预处理逻辑正确。

3.7.2.2 GPU 基础计算验证

执行：

```
bash test_gpu_basic.sh
```

输出如下：

```
vertices: 512
edges: 1536
faces: 1024
interior_edges: 1536
----- Creating SOS Matrix ..... (#fix=15.14) scale = 0.0000000000000010
SoS: matrix[512,2] @ 5 Lia digits; lia_length (10); 0.047 Mb.
jacobi_edges: 64
degenerate_edges: 16
sos_fallback_edges: 16
gpu_kernel_ms: 1.10797
gpu_total_ms: 5.44378
output: /tmp/test_torus_gpu_jacobi.txt
```

该测试说明 CUDA 程序能够完成 Jacobi 集计算，并在 torus 示例上得到 64 条 Jacobi 边。gpu_kernel_ms 表示 CUDA kernel 本身的运行时间，gpu_total_ms 表示包括 GPU 内存分配、数据传输、kernel 执行和结果拷回在内的 GPU 调用时间。

输出中的 degenerate_edges: 16 表示 GPU 判定过程中有 16 条边触发近退化条件。sos_fallback_edges: 16 表示这些边均被 CPU SoS 分支重新计算并覆盖结果。

3.7.2.3 退化边导出验证

执行：

```
bash test_degenerate_dump.sh
```

输出如下：

```
vertices: 512
edges: 1536
faces: 1024
interior_edges: 1536
----- Creating SOS Matrix ..... (#fix=15.14) scale = 0.000000000000010
SoS: matrix[512,2] @ 5 Lia digits; lia_length (10); 0.047 Mb.
jacobi_edges: 64
degenerate_edges: 16
sos_fallback_edges: 16
gpu_kernel_ms: 0.801792
gpu_total_ms: 3.34326
output: /tmp/test_torus_gpu_jacobi.txt
degenerate_dump: /tmp/test_torus_degenerate_edges.txt
```

该测试验证了 --dump-degenerate 功能。程序不仅能够统计退化边数量，还能将退化边列表写入 /tmp/test_torus_degenerate_edges.txt，为调试和分析 SoS 回退提供依据。

3.7.2.4 SoS 回退验证

执行：

```
bash test_sos_fallback.sh
```

输出如下：

```
vertices: 512
edges: 1536
faces: 1024
```

```
interior_edges: 1536
----- Creating SOS Matrix ..... (#fix=15.14) scale = 0.000000000000010
SoS: matrix[512,2] @ 5 Lia digits; lia_length (10); 0.047 Mb.
jacobi_edges: 64
degenerate_edges: 16
sos_fallback_edges: 16
gpu_kernel_ms: 0.959488
gpu_total_ms: 4.23072
output: /tmp/test_torus_gpu_sos_jacobi.txt
```

该结果说明 CPU SoS 回退机制正常工作。GPU 标记出的 16 条退化边均被回退到串行 SoS 逻辑重新计算，最终输出文件为 /tmp/test_torus_gpu_sos_jacobi.txt。

3.7.2.5 小结

以上四个功能测试分别覆盖了：

- (1) CPU 拓扑预处理；
- (2) GPU 并行 Jacobi 判定；
- (3) 退化边检测与导出；
- (4) CPU SoS 精确回退。

测试结果表明，并行政程序的各个功能模块均可正常工作，且在标准 torus 示例上得到的 Jacobi 边数为 64，与串行参考结果一致。

3.7.3 串行与并行输出比对

本节验证并行程序最终输出是否与串行参考程序一致。CUDA 并行程序内部采用“每条边对应一个结果槽位”的方式；若后续改为 GPU 端压缩输出，输出顺序会发生变化。因此，本项目同时使用两种方式验证：

1. 使用 compare_jacobi.py 做顺序无关的边集比较；
2. 使用 diff 直接比较当前输出文件内容。

3.7.3.1 边集比对

执行：

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"
bash test_compare_jacobi.sh
```

输出如下：

```
Reading /mnt/d/金介然/大三下/gpu/大作业/CODE/JacobiSetComputation-master/test_torus.obj... Done.
Computing edges... Done. in 0.145000 msec.
Finding vertex neighbors... Done.
mesh: /mnt/d/金介然/大三下/gpu/大作业/CODE/JacobiSetComputation-master/test_torus.obj
vertices: 512
edges: 1536
faces: 1024
interior_edges: 1536
----- Creating SOS Matrix ..... (#fix=15.14) scale = 0.0000000000000010
SoS: matrix[512,2] @ 5 Lia digits; lia_length (10); 0.047 Mb.
jacobi_edges: 64
degenerate_edges: 16
sos_fallback_edges: 16
gpu_kernel_ms: 0.856064
gpu_total_ms: 3.74022
output: /tmp/test_torus_gpu_sos_jacobi.txt
reference_edges: 64
candidate_edges: 64
match: yes
```

其中：

- (1) reference_edges: 64 表示串行参考结果中有 64 条 Jacobi 边；
- (2) candidate_edges: 64 表示 GPU+SoS 并行结果中也有 64 条 Jacobi 边；
- (3) match: yes 表示两者边集完全一致。

compare_jacobi.py 对每条边只比较顶点编号，并将 (a,b) 规范化为小编号在前，因此比较过程不依赖输出顺序。即使并行程序后续改变输出顺序，只要计算得到的 Jacobi 边集合一致，该测试仍能正确验证结果。

3.7.3.2 文本文件直接比较

执行：

```
diff -u "../JacobiSetComputation-master/test_torus_jacobi.txt" /tmp/test_torus_gpu_sos_jacobi.txt
```

该命令没有任何输出。对于 diff 而言，无输出表示两个文件内容完全一致。因此，当前并行程序生成的 /tmp/test_torus_gpu_sos_jacobi.txt 不仅在边集合上与串行参考一致，而且在文本内容和输出顺序上也与串行文件一致。

3.7.3.3 小结

串行与并行输出比对结果表明：

- (1) 串行参考结果：64 条 Jacobi 边；
- (2) CUDA hybrid 结果：64 条 Jacobi 边；
- (3) 顺序无关边集比较：match: yes；
- (4) diff 文本比较：无差异。

因此，在 torus 标准测试数据上，GPU 并行计算结合 CPU SoS 回退后，最终输出与串行 SoS 版本完全一致。

3.7.4 性能测试与加速比分析

性能测试用于观察串行版本与 CUDA hybrid 版本在不同网格规模下的耗时变化。测试脚本为 benchmark_jacobi.py，其流程为：

1. 使用 mesh_make 自动生成 torus 网格；
2. 运行串行 JacobiSetComputation；
3. 运行 CUDA hybrid 版本；
4. 使用 compare_jacobi.py 比对输出；
5. 将耗时、Jacobi 边数、退化边数、加速比写入 CSV。

3.7.4.1 benchmark 冒烟测试

首先执行小规模冒烟测试：

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"  
bash test_benchmark_smoke.sh
```

输出如下：

```
32x16: cpu_wall=78.414 ms, gpu_wall=312.563 ms, speedup=0.251x, match=yes  
wrote: /tmp/jacobi_benchmark_smoke.csv
```

该测试说明 benchmark 脚本能够正常生成网格、运行串行与并程序、完成正确性比对并输出 CSV。match=yes 表示该规模下 GPU 输出与串行输出一致。

3.7.4.2 多规模性能测试

执行：

```
python3 benchmark_jacobi.py --sizes 32x16,64x32,128x64,256x128,512x256 --output /tmp/jacobi_benchmark.csv
cat /tmp/jacobi_benchmark.csv
```

终端输出如下：

```
32x16: cpu_wall=45.496 ms, gpu_wall=273.897 ms, speedup=0.166x, match=yes
64x32: cpu_wall=93.851 ms, gpu_wall=247.151 ms, speedup=0.380x, match=yes
128x64: cpu_wall=190.336 ms, gpu_wall=287.831 ms, speedup=0.661x, match=yes
256x128: cpu_wall=519.644 ms, gpu_wall=464.860 ms, speedup=1.118x, match=yes
512x256: cpu_wall=1802.050 ms, gpu_wall=1115.213 ms, speedup=1.616x, match=yes
wrote: /tmp/jacobi_benchmark.csv
```

CSV 内容如下：

```
u,v,vertices,edges,jacobi_edges,degenerate_edges,sos_fallback_edges,cpu_wall_ms,gpu_wall_ms,gpu_kernel_ms,gpu_
total_ms,speedup_wall,match
32,16,512,1536,64,16,16,45.496,273.897,0.963584,4.374850,0.166,yes
64,32,2048,6144,128,32,32,93.851,247.151,0.856064,3.649280,0.380,yes
128,64,8192,24576,256,64,64,190.336,287.831,1.000450,3.816610,0.661,yes
256,128,32768,98304,512,128,128,519.644,464.860,1.132030,4.999230,1.118,yes
512,256,131072,393216,1024,260,260,1802.050,1115.213,1.094500,6.059900,1.616,yes
```

为便于阅读，整理如下：

网格规 模	顶点数	边数	Jacobi 边数	退化 边数	CPU 总 时间 (ms)	GPU 端 到端时 间(ms)	GPU kernel(ms)	加速比	比对
32x16	512	1536	64	16	45.496	273.897	0.963584	0.166x	yes
64x32	2048	6144	128	32	93.851	247.151	0.856064	0.380x	yes
128x64	8192	24576	256	64	190.336	287.831	1.000450	0.661x	yes
256x128	32768	98304	512	128	519.644	464.860	1.132030	1.118x	yes
512x256	131072	393216	1024	260	1802.050	1115.213	1.094500	1.616x	yes

3.7.4.3 结果分析

从正确性看，所有规模的 match 均为 yes，表明不同规模下 CUDA hybrid 输出都与串行输出一致。

从性能看，小规模网格下 GPU 端到端时间高于 CPU。例如 32x16 时，CPU 时间为 45.496 ms，GPU 端到端时间为 273.897 ms，加速比为 0.166x。原因是小规模计算量较小，GPU kernel 本身只需约 0.96 ms，但程序整体还包含：

- (1) 网格读取；
- (2) CPU 拓扑预处理；
- (3) CUDA 内存分配；
- (4) CPU 到 GPU 数据传输；
- (5) GPU 到 CPU 结果拷回；
- (6) SoS 矩阵初始化与退化边回退；
- (7) 进程启动与文件输出。

这些固定开销在小规模下占主导，因此端到端加速不明显。

随着网格规模增大，逐边判定的计算量增加，GPU 并行优势逐渐体现。256x128 网格时，GPU 端到端时间为 464.860 ms，CPU 时间为 519.644 ms，加速比达到 1.118x。到 512x256 网格时，CPU 时间为 1802.050 ms，GPU 端到端时间为 1115.213 ms，加速比达到 1.616x。

GPU kernel 时间始终约为 1 ms，即使在 512x256 的 393216 条边规模下，核心判定 kernel 仍然只需约 1.09 ms。这说明被并行化的逐边 Jacobi 判定在 GPU 上执行效率很高。端到端时间主要受 CPU 预处理、I/O、数据传输和 SoS 回退等非 kernel 部分影响。

3.7.4.4 小结

性能测试表明：

- (1) CUDA hybrid 版本在所有测试规模下均保持正确性；
- (2) 小规模时由于固定开销较大，端到端速度慢于串行；
- (3) 随着边数增大，GPU 并行判定优势逐渐显现；

(4) 在 512x256 网格上，端到端加速比达到 1.616x;

(5) GPU kernel 本身耗时很低，说明核心逐边判定已经被有效并行化。

3.7.5 可视化验证

可视化验证用于确认输出的 Jacobi 集在几何形态上符合预期。由于程序输出的是 *_jacobi.txt 文本格式，需要先转换为 ParaView 可读取的 VTK 文件。

3.7.5.1 VTK 文件生成

在 WSL 中进入并行代码目录：

```
cd "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode"
```

执行转换命令：

```
python3 jacobi_to_vtk.py /tmp/test_torus_gpu_sos_jacobi.txt
```

终端输出如下：

```
Wrote /tmp/test_torus_gpu_sos_jacobi.vtk
```

GPU+SoS 输出文件已经成功转换为 legacy VTK 格式。为便于在 Windows 版 ParaView 中打开，将其复制到项目目录：

```
cp /tmp/test_torus_gpu_sos_jacobi.vtk "/mnt/d/金介然/大三下/gpu/大作业/CODE/mycode/test_torus_gpu_sos_jacobi.vtk"
```

生成的可视化文件为：

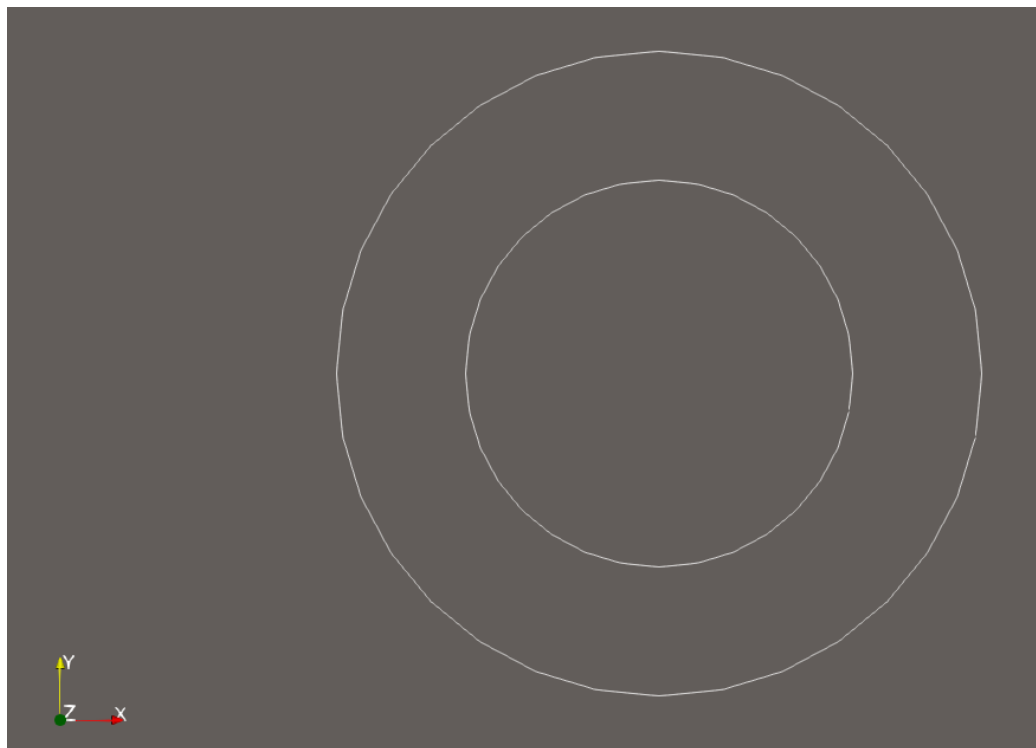
```
CODE/mycode/test_torus_gpu_sos_jacobi.vtk
```

3.7.5.2 ParaView 可视化观察

在 ParaView 中进行了三种观察：

1. 仅加载 `test\_torus\_gpu\_sos\_jacobi.vtk`

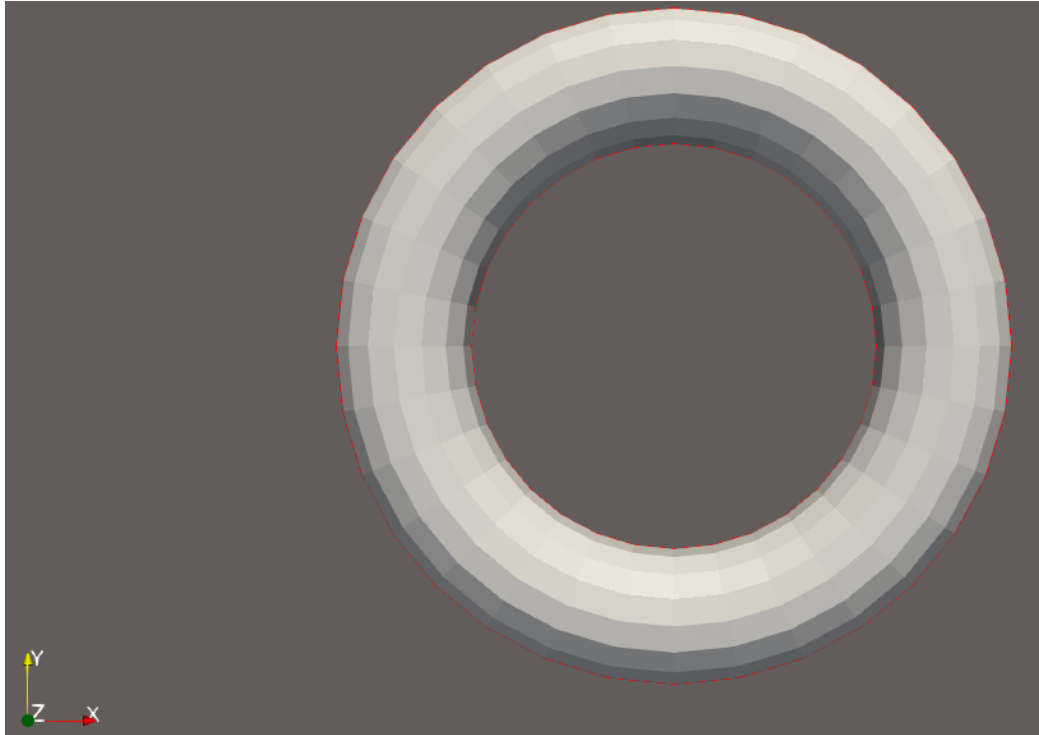
该文件只包含 Jacobi 集线段，不包含原始 torus 曲面。可视化结果显示为平面上的两条同心圆。这与理论预期一致：当前测试函数为 $f=y$ 、 $g=x$ ，在对称 torus 上得到的 Jacobi 集位于 $z \approx 0$ 的截面上，因此从正面看表现为内外两条闭合圆曲线。



仅加载 Jacobi 集 VTK 文件时显示的两条同心圆

2. 同时加载 `test\_torus.obj` 和 `test\_torus\_gpu\_sos\_jacobi.vtk` 的正面图

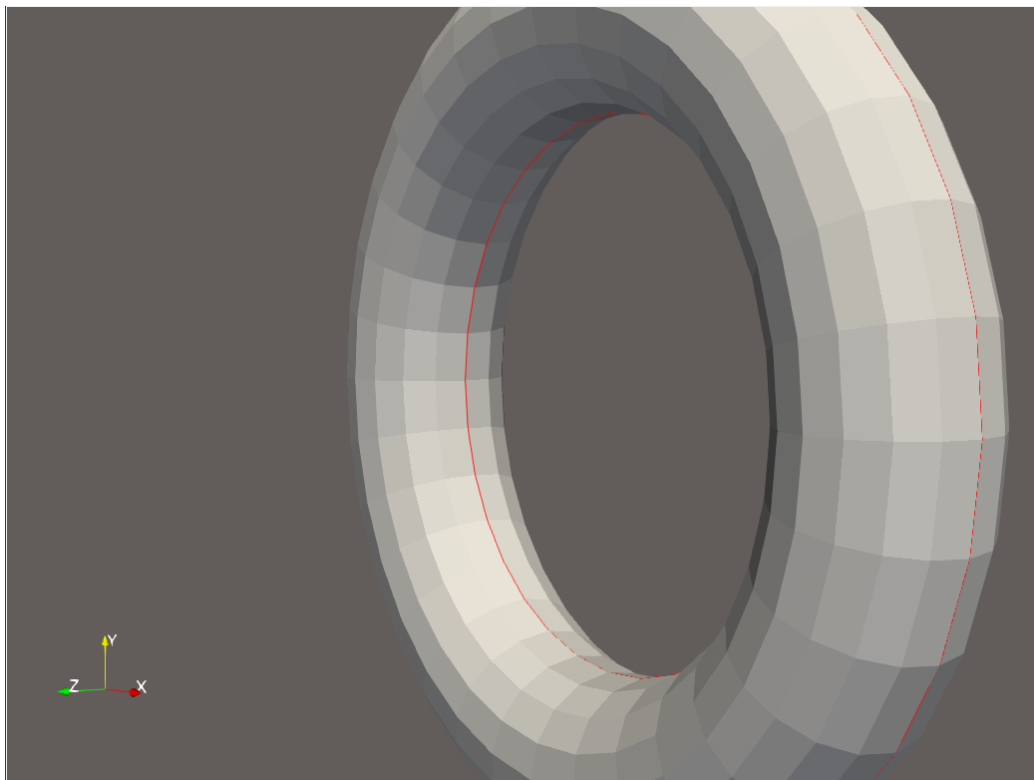
在正面视角下，Jacobi 集曲线叠加在 torus 上。由于视角沿着 torus 轴向观察，红色 Jacobi 曲线与圆环边界视觉上较接近，因此同心圆结构不如单独打开 .vtk 时明显。但 Jacobi 曲线与 torus 几何位置一致，位于圆环的内外圈附近。



Jacobi 集与 torus 叠加后的正面图

3. 同时加载 `test\_torus.obj` 和 `test\_torus\_gpu\_sos\_jacobi.vtk` 的侧面放大图

侧面放大观察时，红色 Jacobi 曲线更加清晰。曲线贴合在 torus 表面，并分别位于圆环外侧和内侧的环向位置。这表明并程序输出的 Jacobi 边不仅在文本结果上与串行版本一致，在几何空间中也落在正确的网格表面位置。



Jacobi 集与 torus 叠加后的侧面放大图

从可视化形态看，GPU+SoS 版本输出的 Jacobi 集满足以下特征：

- (1) 单独显示时为两条闭合曲线；
- (2) 从正面看呈两条同心圆；
- (3) 与 torus 曲面叠加后，曲线位于圆环表面的内外两圈；
- (4) 侧面放大后，红色曲线贴附在 torus 表面，没有出现明显偏移或断裂。

3.7.5.3 小结

可视化验证表明，CUDA hybrid 程序输出的 Jacobi 集不仅在数值比对中与串行结果一致，而且在几何形态上符合预期。对于 $f=y$ 、 $g=x$ 的 torus 示例，Jacobi 集表现为 $z \approx 0$ 截面上的两条闭合圆曲线，叠加到 torus 曲面后对应圆环的内外两圈。该结果与前面文本比对和性能测试共同验证了并行实现的正确性。

实验总结

本次实验围绕论文理解、串行代码阅读和 CUDA 并行化实现，完成了 Jacobi 集算法的工程化改造。论文中的 Jacobi 集涉及 Morse 函数、流形、临界关系和 lower link 等概念；串行项目将这些理论转化为三角网格上的具体边判定，使抽象算法具备可调试和可验证的程序形式。通过分析 `JacobiSet::compute()`、`POS1`、`POS2` 和 `is_lowerLink()`，论文中的局部判定思想最终落到每条内部边及其 link 顶点上。该局部性为 CUDA 并行化提供了明确切入点：每条边的判断相互独立，能够自然映射为一个 GPU 线程。因此，并行化的关键不在于简单地把代码放入 `.cu` 文件，而在于找到算法中真正独立、重复、规模较大的计算单元。

实现过程中，正确性是并行化设计的首要约束。Jacobi 集计算存在浮点退化问题，串行项目使用 SoS 保证符号判定稳定。若只实现普通 double 精度 GPU 判定，结果会在退化边上产生与串行参考不一致的风险。因此，本实验采用 CPU-GPU 混合方案：CPU 负责网格读取、拓扑预处理和 SoS 回退，GPU 负责大规模普通边的并行判定。测试结果表明，小规模下端到端性能会受到启动、传输、预处理和 SoS 初始化开销影响；随着网格规模增大，GPU 并行判定的优势逐渐体现。最终程序在保持与串行结果一致的前提下，在大规模网格上获得一定加速，验证了该混合并行方案的有效性。

本实验也体现了异构并行政程序的设计取舍。GPU 擅长处理结构规则的大量数值计算，但不适合直接承载所有复杂控制逻辑和动态数据结构。合理方案是将 CPU 和 GPU 各自擅长的部分组合起来，并通过清晰的数据接口连接。本项目中的 `EdgeRecord`、`JacobiGpuResult`、比对脚本和 `benchmark` 脚本，使并行实现具备可运行、可验证和可扩展的工程基础。后续优化方向包括减少 CPU 预处理开销、改进 GPU 数据布局、支持外部标量场输入，以及研究 SoS 判定在 GPU 上的实现方案。